GiraffeGraph bitirme projesi raporu

Büyük Dil Modellerinde Halüsinasyon ve Şema Sapmasını Önleyen İnsan-Döngülü Bilgi Grafı Üretim Motoru: Hadis İsnad Zincirleri Üzerine Bir Uygulama
Özet
1 Giriş
- 1.1 Problem
- 1.2 Vaka çalışması
- 1.3 Amaç ve yaklaşım
- 1.4 Katkılar
- 1.5 Kapsam ve sınırlar
- 1.6 Yapı

2 Arka plan
- 2.1 Hadis, isnad ve metin
- 2.2 Bilgi grafları ve şemayla sınırlandırılmış çıkarım
- 2.3 İki hata türü: kaynağın desteklemediği olgular ve şema sapması
- 2.4 Kaynağa dayandırma ve kaynakla ilişkilendirme
- 2.5 İnsan-döngülü inceleme ve yönlendirme sinyali olarak güven skoru
- 2.6 Türetilebilir olgular ve kararlar

3 İlgili çalışmalar
- 3.1 Hadis derlemleri ve etiketli kaynaklar
- 3.2 İsnad ve ravi çıkarımı
- 3.3 Ravi kimlik çözümleme
- 3.4 Hadis bilgi grafları ve model tabanlı yapılandırma
- 3.5 Model tabanlı bilgi grafı kurma ve değerlendirme
- 3.6 Alan derlemeleri
- 3.7 Çalışmanın konumu

4 Gereksinimler ve kapsam
- 4.1 Kullanıcılar ve kullanım senaryoları
- 4.2 İşlevsel gereksinimler
- 4.3 İşlevsel olmayan gereksinimler
- 4.4 Kapsam dışı

5 Sistem tasarımı
- 5.1 Tasarım ilkeleri
- 5.2 Mimari
- 5.3 Veri modeli
- 5.4 Adım adım boru hattı
- 5.5 Doğrulama kontrolleri
- 5.6 İlişki kimliği
- 5.7 Karar kaydı
- 5.8 İsnad vaka çalışmasının eklentileri
- 5.9 Tasarım kararları ve alternatifler

6 Gerçekleştirim
- 6.1 Teknoloji yığını
- 6.2 Depo düzeni
- 6.3 API arayüzü
- 6.4 Model sağlayıcısı soyutlaması
- 6.5 Kalıcı saklama

- 6.6 Panel
- 6.7 Testler ve sürekli entegrasyon
- 6.8 Çalıştırma
- 6.9 Bilinen sınırlamalar

7 Değerlendirme
- 7.1 Sorular
- 7.2 Veriler ve referans etiketleri
- 7.3 Protokol
- 7.4 Koşullar
- 7.5 Çıkarım sonuçları
- 7.6 Kontrollerin yakaladıkları
- 7.7 Modelin güven skoru
- 7.8 İnceleme yükü, commit işlemleri ve maliyet
- 7.9 İkinci bir model: yapılandırma hatası
- 7.10 Ravi kimliği
- 7.11 Ölçmediklerimiz
- 7.12 Önceki pilot

8 Destek çalışmaları: neyin saklanması gerekir
- 8.1 Etiketli isnad zincirlerinde kimlik kararının yeniden kullanımı
- 8.2 İkinci bir alan: kod ajanı için neyin saklanması gerekir
- 8.3 İki çalışmanın ekledikleri ve göstermedikleri

9 Tartışma
- 9.1 Kontrolün yeri
- 9.2 Modele ne için ihtiyaç var?
- 9.3 Güven skoru
- 9.4 Katı commit kuralının maliyeti
- 9.5 Saklanan kararlar ve türetilen olgular
- 9.6 Geçerlilik tehditleri
- 9.7 Sistemin sınırları

10 Sonuç ve gelecek çalışma
- 10.1 Bitirme Projesi II planı
- 10.2 Hadis dışındaki alanlar

Kaynaklar

Ek A Sonuçları yeniden üretme

Ek B Sonuç dosyaları

Ek C Yapay zekâ araçlarının kullanımı

Büyük Dil Modellerinde Halüsinasyon ve Şema Sapmasını Önleyen İnsan-Döngülü Bilgi Grafı Üretim Motoru: Hadis İsnad Zincirleri Üzerine Bir Uygulama

Bitirme Projesi Raporu

Yahya Efe Kuruçay, Deniz Büyükşahin, Alperen Ulukaya

Danışman: Dr. Taha Yiğit Alkan

Akdeniz Üniversitesi, Mühendislik Fakültesi, Bilgisayar Mühendisliği Bölümü

CSE 491 Bitirme Projesi I, Güz 2026–2027

13 Ekim 2026 taslağı

Özet

Büyük dil modelleri (BDM) metni bilgi grafı olgularına dönüştürebilir; ancak kaynakta olmayan olgular ekler ve kendilerine verilen şemadan sapar. Bu raporda, modelin yalnızca öneride bulunduğu GiraffeGraph Core motorunu sunuyoruz. Önerilen her olgu kaynaktan bir alıntı taşır. Deterministik kontroller, bir insanın olguyu incelemesi gerekip gerekmediğini belirler. Olgu, ancak bir inceleyicinin kararı ya da adı kayıtlı bir otomatik politika aracılığıyla kabulü kaydedildiğinde grafa girer. Vaka çalışmamız, hadisin başında yer alan ravi zinciri, yani isnaddır. Tasarım tek bir iddiaya dayanır: bir dil modeliyle kurulan bilgi katmanının saklaması gereken şey insanın verdiği karardır; kaynaktan türetilebilen her şey yeniden türetilir.

Motoru, altı temel hadis kitabından 300 hadis üzerinde değerlendirdik. Mevcut bir platform kaynaklı derlemin ravi etiketlerini referans olarak kullandık; eşleştirme protokolünü sabit tuttuk ve hadis düzeyinde bootstrap aralıkları hesapladık. Gevşek eşleştirmede kural tabanlı ayrıştırıcı F1 0,923'e, yalnız şema verilen model 0,890'a, tip açıklamaları ve alan ipuçları eklenen aynı model ise 0,956'ya ulaştı. Katı eşleştirmede her iki model koşulu da ayrıştırıcının üzerindeydi. Çıkarılan adın modelin kendi alıntısında geçip geçmediğine bakan kontrol, yalnız şema verilen modelin önerilerinin %9'unu işaretledi. İşaretlenen önerilerin %79'u referansla eşleşmiyordu ve bunlar modelin etiketle eşleşmeyen tüm önerilerinin %84'ünü oluşturuyordu. Çoğu, modelin çevriyazıya çevirdiği, başka bir ad hâlinde yazdığı ya da bir akrabalık ifadesinden çıkarsadığı adlardı. İncelemesiz kabul edilen adaylarda precision 0,914'ten 0,985'e çıktı. Alıntının kaynakta geçip geçmediğine bakan kontrol hiçbirini bulamadı; çünkü model doğru alıntı yapıp adı değiştiriyordu. Modelin bildirdiği güven skoru, etiketle eşleşen ve eşleşmeyen önerileri zayıf ayırdı (AUC 0,634). Raviyi yazılı adıyla tanımlamak, her on zincir konumundan birini birden çok kişinin paylaştığı bir ad altında topladı.

İki destekleyici çalışmada iddianın saklamayla ilgili tarafını sınadık. Etiketli 27.056 zincir üzerinde yapılan bir benzetimde, anahtarı yalnızca yazılı ad olan kimlik kararı her on geçişten birinde yanlış yeniden kullanıldı. Kararın kapsamını deterministik bir kuralın saptadığı çelişkilerden türetmek, sabit ve dar bir anahtarın gerektirdiği inceleyici sorularının yaklaşık yarısıyla sessiz hatayı geçiş başına 0,014'e indirdi. Sonuç, ikinci bir etiket kümesinde ve dışarıda bırakılan kitaplarda tekrarlandı. İkinci bir alanda, kod ajanı motorun kodunun neden o şekilde yazıldığına ilişkin soruları yalnız koddan %0 ile %4 arasında, karar defteriyle %88 ile %100 arasında doğru yanıtladı. Yeterli araç çağrısı hakkı olduğunda ise yapısal olguları hiçbir belge olmadan koddan okuyabildi.

Referans etiketleri insan altın standardı değildir; koşuları hiçbir insan incelememiştir. Motorun değerlendirmesinde kullanılabilir sonuç üreten tek bir model vardır. Defter çalışmasındaki inceleyici etiketlerden benzetilmiştir ve deneyler motorun bellekte çalışan arka ucunda yürütülmüştür. Yazılımda 285 otomatik arka uç testi vardır; bunların on ikisi boru hattını PostgreSQL ve Neo4j üzerinde sınar. Yazılım ayrıca, bir zincirin her bağlantısına ait kaynak alıntısı ve kabul kaydıyla birlikte okunabildiği bir panel içerir.

Anahtar kelimeler: bilgi grafı, büyük dil modelleri, insan-döngülü sistem, kaynağa dayandırma, karar kaydı, şema sapması, hadis, isnad

1 Giriş

1.1 Problem

Metinden kurulan bir bilgi grafı, ancak içerdiği her olguya güvenilebiliyorsa ve her olgu izlenebiliyorsa yararlıdır. Büyük dil modelleri (BDM) çıkarımı ucuzlatır: bir metin parçası ve bir şema verildiğinde model, saniyeler içinde düğüm ve ilişkiler döndürür. Aynı modeller, sonradan fark edilmesi zor iki tür hata yapar. Metin parçasının desteklemediği olgular döndürür ve kimsenin tanımlamadığı bir tip ya da özellik kullanarak şemadan sapar. Böyle bir çıktı grafa yazıldığında doğru çıktıdan ayırt edilemez.

Yaygın çözüm, modelden bir alıntı ve güven skoru istemektir. Alıntı, olgunun nereden geldiğini gösterir; skor ise hangi olguların kontrol edileceğine işaret eder. Biz de aynı düşünceden yola çıktık. Kendi kodumuzda, bir pilot çalışmada ve bu raporun deneylerinde, denediğimiz modellerle ikisinin de varsayıldığı gibi çalışmadığını gördük. Model, kaynağı doğru alıntılıyor, sonra alıntıda olmayan bir değer yazıyordu. Referansla eşleşen ve eşleşmeyen çıktılara verdiği güven skoru neredeyse aynıydı.

1.2 Vaka çalışması

Problemi, hadisin başında yer alan ravi zinciri, yani isnad üzerinde inceliyoruz: "A bize anlattı, o B'den, o C'den". Bu görev, sorumuza üç nedenle uygundur. Zincir, doğası gereği bir graftır. Hatalar sonucu etkiler; çünkü âlimler hadisi zincirindeki kişilere göre değerlendirir ve tek bir yanlış ad hükmü değiştirir. Son olarak, etiketli veri vardır; dolayısıyla çıktıyı gözle incelemek yerine puanlayabiliriz.

Bu görev çözümsüz değildir. Büyük etiketli eğitim kümeleri bulunduğunda ince ayar yapılmış modeller ravileri ve aralarındaki bağlantıları doğru biçimde çıkarır [1]. Altı temel hadis kitabının ontolojisi ve bilgi grafı yayımlanmıştır [2, 3]; ravi adları da biyografik veritabanlarına bağlanmıştır [4]. Bölüm 3'te bu çalışmaları inceliyoruz. Bulamadığımız şey, genel amaçlı bir model ile graf arasındaki adımı inceleyen bir çalışmadır: bu adımda hangi kontroller yer almalı, her biri neyi yakalar ve bir olgunun kabulü nasıl kaydedilir?

1.3 Amaç ve yaklaşım

Amacımız, dil modelinin graf olguları önerdiği, deterministik kontrollerin ve insan incelemesinin hangilerinin kabul edileceğini belirlediği, kabul edilen her olgunun da kaynak alıntısını ve nasıl kabul edildiğine ilişkin kaydı koruduğu bir motordur. İkinci amacımız, kontrollerin katkısını ölçmektir.

Her iki amacın arkasında, Bölüm 2.6'da bütünüyle ifade ettiğimiz tek bir iddia vardır. Ayrıştırıcının ya da modelin metinden çıkardığı bir olgu yeniden çıkarılabilir; bu yüzden saklanması gerekmez. Bir insanın kabulü, reddi ya da düzeltmesi ise hiçbir şeyden türetilemez ve bir kez saklanmalıdır. Motoru bu ayrım üzerine kurduk. Üçüncü amacımız da bu ayrımı motorun kendi görevinde ve başka bir alanda sınamaktır.

GiraffeGraph Core motoru, modeli güvenilmeyen bir öneri üreticisi olarak ele alır. Bir kontrolden geçemeyen aday sessizce reddedilmez; gerekçe koduyla inceleme kuyruğuna gider. İnceleyicinin kararı, inceleyicinin adı, zaman ve süreyle birlikte saklanır; sonraki işlemler bu kararı geçersiz kılamaz. İnsan müdahalesi olmadan kabul edilen adaylar, adı kayıtlı bir politika tarafından kabul edilmiş olarak kaydedilir. Hadise özgü davranışlar eklentilerle eklenir: bir bölütleme stratejisi, bir şema şablonu ve bir kural tabanlı sağlayıcı.

1.4 Katkılar

1. **Çalışan bir motor.** Motor, kaynağa dayandırılmış adaylar, adayı incelemeye yönlendiren on gerekçe kodu ve tanımsız tip ve özellikleri kaydeden iki gerekçe kodu içerir. Aynı iki ravi arasındaki ayrı aktarımları ayrı tutan ilişki kimliği, yalnızca yeni kayıt eklenen bir karar kaydı, zincir görünümü ve gerçek PostgreSQL ve Neo4j örnekleri üzerinde de çalışan bir test paketi vardır (Bölüm 5 ve 6).
2. **Motor içindeki kontrollerin ölçümü.** Daha önce görülmemiş 300 hadiste, değer-alıntıda kontrolü, yalnız şema verilen modelin referansla eşleşmeyen 145 önerisinin 122'sini, henüz hiçbiri kabul edilmeden işaretler. Alıntı-kaynakta kontrolü ise hiçbirini işaretlemez (Bölüm 7).
3. **Çıkarım koşullarının tek bir boru hattında karşılaştırılması.** Kurallar, yalnız şema verilen model, tip açıklamaları ve alan ipuçları eklenen model ve girdisi zincir metin aralığıyla sınırlandırılan aynı model, eşli bootstrap aralıklarıyla karşılaştırılır. Yalnız şema içeren isteme açıklama ve ipucu eklemek F1'i 0,066 artırır; girdiyi zincir metin aralığıyla sınırlandırmak yarar sağlamaz.
4. **Bir değerlendirme kiti ve veri kümeleri.** Sabit eşleştirme protokolü kullanan bir puanlayıcı, kontrollerin, güven skorunun ve maliyetin analizi, mevcut etiketli bir derlemden oluşturulmuş ve kökeni belirtilmiş 300 hadislik test kümesi ve gelecekte hazırlanacak insan etiketli küme için kılavuz içeren bir etiketleme formu sunuyoruz.
5. **Ravi kimliği için bir taban çizgisi.** Bu taban çizgisi, referans etiketlerinde yazılı bir adın ne sıklıkla birden çok kişiyi temsil ettiğini sayısallaştırır.
6. **İki etiketli derlem üzerinde ravi kimliği karar defteri benzetimi.** Anahtarı yalnızca ad olan bir karar, her on geçişten birinde yanlış yeniden kullanılır. Kararın kapsamını deterministik bir kuralın saptadığı çelişkilerden türetmek, sabit ve dar bir anahtarın hata düzeyine, onun gerektirdiği inceleyici sorularının yaklaşık yarısı ile üçte ikisi arasında kalan bir soru sayısıyla yaklaşır. Bu sonuç, defterin görmediği kitaplarda da elde edilir (Bölüm 8.1).
7. **İkinci bir alanda deney.** Bu deneyde bir kod ajanı, motorun kaynak kodu hakkındaki soruları yanıtlar. Kararlara ilişkin soruları yalnız koddan neredeyse hiç yanıtlayamazken karar defteriyle neredeyse her zaman yanıtlar. Kodu okuyabilecek kadar kaynak ayırabildiğinde, kodun saklanmış bir açıklamasına ihtiyaç duymaz (Bölüm 8.2).

1.5 Kapsam ve sınırlar

Rapor bir prototipi anlatır. Ravi kimlik çözümleme, biyografik tarihler gerektiren kurallar, graf üzerinden soru yanıtlama ve kimlik doğrulama motorun kapsamı dışındadır (Bölüm 4). Kimlik kararları ve tarih kuralı yalnızca Bölüm 8.1'deki benzetimde yer alır. Referans etiketleri mevcut derlemlerden gelir; bağımsız etiketleyicilerin ürettiği bir altın standart değildir. Değerlendirme koşularını hiçbir insan incelememiştir; dolayısıyla döngünün insan tarafı uygulanmış ve test edilmiş, ancak ölçülmemiştir. Motorun değerlendirme sonuçları, bellekte çalışan arka uç üzerinde tek bir modelden elde edilmiştir. İki destekleyici çalışma keşif niteliğindedir: biri inceleyiciyi etiketlerden benzetir, diğeri okuyucu olarak bir model kullanır. Bölüm 7'de ölçülmeyenleri listeliyor, Bölüm 9'da sonuçlara ne ölçüde güvenilebileceğini tartışıyoruz.

1.6 Yapı

Bölüm 2'de hadis zincirlerine ve dil modelleriyle çıkarıma ilişkin arka planı veriyoruz. Bölüm 3'te ilgili çalışmaları inceliyoruz. Bölüm 4, gereksinimleri ve kapsamı belirler. Bölüm 5 ve 6, tasarımı ve gerçekleştirimini anlatır. Bölüm 7, motorun değerlendirmesini raporlar. Bölüm 8'de nelerin saklanması gerektiğine ilişkin iki destekleyici çalışmayı sunuyoruz. Bölüm 9'da sonuçları tartışıyor, Bölüm 10'da ikinci dönem planıyla raporu tamamlıyoruz.

2 Arka plan

Bu bölümde vaka çalışmasının alanını ve tezin geri kalanının dayandığı kavramları tanıtıyoruz.

2.1 Hadis, isnad ve metin

Hadis iki parçadan oluşan bir rivayettir. İsnad (إسناد), rivayetin aktarıldığı ravilerin zinciridir; metin (متن) ise rivayet edilen sözün kendisidir. Bazı kaynaklar ve veri kümesi adları aynı zincir için sened sözcüğünü kullanır. Bundan sonra isnad için "zincir", metin için "rivayet metni" yazıyoruz.

Zincir, kitabı derleyen kişiden başlayarak yazılır ve Hz. Muhammad'e ya da ilk ravi kuşağı olan sahabilerinden birine kadar geriye gider. Zincirde adı geçen her ravi, rivayeti kendisinden sonra adı geçen raviden almıştır. Örneğin Sanadset derleminde al-Nasai'nin kitabındaki 693 numaralı hadis için etiketlenen zincirde beş ravi vardır: Muhammad ibn Mansur, Sufyan, al-Zuhri, Sa'id ve Abu Hurayra [5].

Raviler, aktarım kalıplarıyla (sigha, صيغة) birbirine bağlanır. Yaygın kalıplar arasında haddathana ("bize rivayet etti"), akhbarana ("bize haber verdi"), sami'tu ("işittim") ve 'an (عن, "-den") vardır. Kalıp, rivayetin bir raviden diğerine nasıl geçtiğini belirtir; 'an ise bunu açık bırakır. Bu yüzden graf modeli, kalıbı iki ravi arasındaki ilişkinin bir özelliği olarak tutar.

Bir hadis, derleyiciye birden çok zincir üzerinden ulaşabilir. Bu paralel zincirler art arda yazıldığında, tahvil (تحويل) denen ح harfi yeni bir zincire geçişi belirtir; buna zincir geçiş işareti diyoruz. Altı temel hadis kitabından seçtiğimiz 300 hadislik örneklemde, kaynak etiketlerine göre 74 hadisin birden çok zinciri vardır [6]. Bir hadisin zincirleri ayrı tutulmazsa grafta hiçbir zincirin belirtmediği aktarım yolları oluşur.

Zor olan ravi kimliğidir. Aynı kişi "Sufyan ibn Uyayna", "Ibn Uyayna" ya da "Sufyan" olarak geçebilir; farklı kişiler de aynı kısa adı paylaşabilir. Yukarıdaki zincirde metin, hangi Sufyan'ın ya da hangi Sa'id'in kastedildiğini söylemez. "Babasından, dedesinden" gibi bazı zincirlerde hiç kimsenin adı verilmez.

Her adın bir ravi kimliği taşıdığı AR-Sanad test kümesindeki 27.056 gerçek zincirde, 15.301 farklı ad dizgisinin 815'i birden çok kimliğe karşılık gelir. Muhammad adı 27 raviye, "babası" ifadesi ise 421 raviye karşılık gelir [6, 7].

Kimlik gereklidir; çünkü zincir, ravileri üzerinden değerlendirilir. Rical (رجال) denen ravi biyografisi literatürü, her ravinin kim olduğunu ve âlimlerin onu ne kadar güvenilir bulduğunu kaydeder. Kuşaklara göre düzenlenmiş eserler bu literatürün bir parçasıdır [8]. Kullandığımız veri kümeleri, tabaka ve Ibn Hajar ile al-Dhahabi'ye atfedilen güvenilirlik dereceleri gibi bu geleneğe ait alanlar taşır [6]. İki farklı raviyi tek düğümde birleştirmek zincir hakkındaki hükmü değiştirebilir; birleştirmenin atlanması ise yalnızca yinelenen bir düğüm bırakır. Metin tek başına bu durumları çözemez; bunlar için ravilerin yetkili bir kimlik listesine ve bir uzmana ihtiyaç vardır.

Altı temel hadis kitabı (Türkçede Kütüb-i Sitte), al-Bukhari, Muslim, Abu Dawud, al-Tirmidhi, al-Nasai ve Ibn Majah'ın kitaplarıdır. Eleştirel bir derleme, değerlendirme ölçütlerinin bu altı kitaba dar biçimde odaklanmasını alanın zayıflıkları arasında sayar [9].

2.2 Bilgi grafları ve şemayla sınırlandırılmış çıkarım

Bilgi grafıyla, tipli ilişkilerle birbirine bağlanmış tipli düğümler kümesini kastediyoruz; düğümler ve ilişkiler özellik taşır. Kavrama ve alana özgü graf kurulumuna ilişkin derlemeler [10, 11] kaynaklarında bulunabilir. Şema, hangi düğüm tiplerine, ilişki tiplerine ve özelliklere izin verildiğini belirler. Ravi zincirleri için bir

şemada az sayıda öğe yeterlidir: ravi düğümü, hadis düğümü ve hadis referansını, zincir numarasını, zincirdeki konumu ve aktarım kalıbını taşıyan bir "-den rivayet etti" ilişkisi.

Graf bu alana uygundur; çünkü zincirler hakkındaki sorular yol ve ağ sorularıdır. İki zincirin ortak ravisi herhangi bir tek metin parçasında yazmaz; onu bulmak için her iki zinciri de dolaşmak gerekir.

Büyük dil modelleri (BDM), metinden bu tür graflar kurabilir [12, 13]. İki strateji vardır. Önce şemanın belirlendiği çıkarımda model, izin verilen tipleri ve bir çıktı sözleşmesini alır; çıktısı şemaya göre doğrulanır. İkinci stratejide model serbestçe çıkarım yapar, sonraki bir adım ise ürettiklerini tanımlar ve kanonikleştirir. Extract, Define, Canonicalize çerçevesi buna örnektir [14]. Bu yaklaşım önceden şema gerektirmez. Bizim değerlendirmemize göre bunun bedeli ikinci bir geçiştir; kanonikleştirme de hataların oluşabileceği bir adım daha ekler.

2.3 İki hata türü: kaynağın desteklemediği olgular ve şema sapması

Projenin başlığı iki hata türünü adlandırır. Şema sapması, çıktının şemada tanımlanmayan tipler ya da özellikler kullanmasıdır. Aynı ilişki bir grupta HEARD_FROM, sonraki grupta TOLD ya da narratedBy olarak adlandırılabilir. Tip düzeyinde model, tanımsız bir düğüm ya da ilişki tipi yazar. Özellik düzeyinde ise tanımsız bir özellik anahtarı ya da izin verilen listenin dışında bir değer yazar.

Halüsinasyon, kaynağın desteklemediği üretilmiş içerik için kullanılan yaygın addır. Ji ve arkadaşları, doğal dil üretimindeki halüsinasyonları derler [15]. Bu derleme, bilgi grafı çıkarımını bir görev olarak ele almaz; dolayısıyla verdiği sayılar bizim düzenimize aktarılamaz. Biz daha yalın olan "kaynağın desteklemediği olgu" terimini kullanıyoruz.

Tek bir "halüsinasyon oranı", çıkarım için fazla genel kalır; çünkü bu başlık altında en az üç farklı durum sayılır:

- **Değer kaynakta yoktur.** Çıkarılan değer, örneğin bir ravi adı, kaynak metinde hiç geçmez.
- **Değer modelin kendi alıntısında yoktur.** Model, kaynakta gerçekten geçen bir parçayı alıntılar; ancak bildirdiği değer o parçada geçmez. Bu durum, gerçek hataları ve sessiz normalizasyonu kapsar. Örneğin metinde belirtme hâlinde bulunan bir ad, yalın hâlde döndürülmüş olabilir. İkinci durumda model, hiçbir yerde kaydedilmeyen bir karar vermiştir.
- **Kategori hatası.** Değer metinde geçer ve tip şemada vardır; ancak çıkarılan öğe o tipe ait değildir. Örneğin Peygamber'in ravi olarak çıkarılması böyledir.

Bu üç sınıfı farklı kontroller yakalar; bu yüzden Bölüm 7.6'da ayrı ayrı raporluyoruz. Text2KGBench, metinde bulunmanın neden doğru olmakla aynı şey olmadığını gösterir. Halüsinasyon ölçüsü, bir üçlünün öznesinin ya da nesnesinin cümlede veya ontolojide bulunup bulunmadığını sınar. Değerlendirme ölçütünün Wikidata-TekGen bölümünde Vicuna-13B ile özne halüsinasyonu 0,17 iken referans üçlülere karşı precision 0,38 olmuştur [16].

2.4 Kaynağa dayandırma ve kaynakla ilişkilendirme

Kaynağa dayandırma, her çıktıyı kaynaktaki kanıta bağlar. İki yöntem ailesi burada ilgilidir.

Alıntı tabanlı kontroller, modelden her aday olguyla birlikte birebir alıntı döndürmesini ister ve ardından dizgi testleri uygular: alıntı kaynak segmentte geçiyor mu, çıkarılan değer alıntıda geçiyor mu? Bu testler deterministiktir, maliyetleri yok denecek kadar azdır ve her adayda çalıştırılabilir. Bir şeyin yanlış olduğunu gösterebilirler; adayın doğru olduğunu gösteremezler.

İddia düzeyindeki yöntemler, serbest metnin değerlendirilmesinden gelir. FActScore, uzun bir yanıtı atomik iddialara ayırıp her birini doğrular; SAFE de uzun metinlerin olgusal doğruluğunu benzer biçimde ele alır [17, 18]. Rashkin ve arkadaşları, üretilen metnin belirlenmiş kaynaklara dayandırılabilirliğinin nasıl ölçüleceğini

tanımlar [19]. SummaC ve AlignScore, çıkarımsal gerektirme ya da hizalama modelleri kullanarak bir iddianın bir metinle tutarlılığını puanlar [20, 21]. RARR ve Chain-of-Verification, çıktıyı düzeltmek ya da doğrulamak için ek model çağrıları kullanır [22, 23]. Bu yöntemler cümle ya da iddia düzeyinde çalışır; otomatik olanlar öğe başına en az bir model çağrısı gerektirir. Ravi adı gibi kısa, yapılandırılmış bir değer için dizgi testleri daha ucuz bir ilk adımdır. Anlamı değerlendiremedikleri için ardından bir insan adımı gelir.

2.5 İnsan-döngülü inceleme ve yönlendirme sinyali olarak güven skoru

İnsan-döngülü tasarımda bir insan, aday olgular grafa yazılmadan önce onlar hakkında karar verir. Uzman zamanı kıt kaynaktır; bu yüzden sistemin, insanın görmesi gereken adayları seçen bir yönlendirme sinyaline ihtiyacı vardır.

İlk akla gelen sinyal, modelin kendi çıktısı için bildirdiği güven skorunu bir eşikle karşılaştırmaktır. Modelin bildirdiği bu güven skorunun doğrulukla örtüşüp örtüşmediği, kalibrasyon problemi olarak incelenir [24, 25]. Hadis alanı, eğitilmiş bir sınıflandırıcıda bile ihtiyat gerektirir: Mahmoud ve arkadaşları, ravi tanıma modellerinin yanlış tahminlerinin %12,9'unun 0,90'ın üzerinde güven skoruyla yapıldığını bildirir [7]. Bölüm 2.6'da ele aldığımız deneme yazısı, modelin hatalarının dışarıdan doğru çıktısına benzediğini savunur [26].

İki alternatif vardır. Bölüm 2.4'teki bir testin başarısız olması gibi deterministik risk işaretleri, modelin kendisi hakkındaki değerlendirmesini kullanmadan adayı incelemeye yönlendirebilir. Konformal tahmin, bir kalibrasyon kümesi bulunduğunda dağılımdan bağımsız hata güvenceleri sunar [27]. Sinyal ne olursa olsun, iki yönlendirme politikası ancak aynı kapsama oranında karşılaştırılabilir. Raja, seçici yanıt sunmanın hata oranlarını bu şekilde karşılaştırır [28]. Bölüm 7.7'de güven skoru sinyalini bir kontrolle, incelemeye gönderilen geçişlerin karşılaştırılabilir paylarında karşılaştırıyoruz.

2.6 Türetilebilir olgular ve kararlar

Son kavram, projenin "The Cache and the Ledger" adlı iç deneme yazısından gelir [26]. İlkesi şudur: doğruluğun esas alındığı bir kaynaktan türetilebilen olgu önbellektir ve saklanmak yerine yeniden üretilmelidir; türetilemeyen olgu ise karardır ve bir kez saklanmalıdır.

Deneme yazısının örneği bir belgelendirme sayfasıdır. "Denetleyici bu dosyadadır" bilgisi koddan yeniden elde edilebilir. "Başka bir tasarımı düşündük ve reddettik" bilgisi ise elde edilemez; çünkü kod, seçilmeyen seçeneği içermez.

Deneme yazısı, ilkeyi kendi betiklerinin yeniden ürettiği sayımlarla destekler. 733 Python Enhancement Proposals belgesi içinde, bir linter başlığı kontrol ettiği hâlde 42 ileri yönlü "replaces" bağlantısının 27'sine geri bağlantı verilmiştir (%64). Yazı, saklanan türetilebilir değerlerin zamanla bozulduğu, geri bağlantıları türetilen ve yalnızca yeni kayıt eklenen karar kayıtlarının ise tasarımları gereği bu sorunu önlediği sonucuna varır. Sınırları da belirtir: kararların yerleşik bir sağlama toplamı yoktur ve otomatik bir kontrol problemi işaretleyebilir, ancak bir ifadenin doğru olduğunu onaylayamaz.

Çıkarıma uyguladığımızda: ayrıştırıcının ya da modelin metinden çıkardığı aday olgu türetilebilirdir; çünkü model her seferinde aynı çıktıyı vermese de aynı metinden yeniden çıkarılabilir. Bir insanın kabulü, reddi, düzeltmesi ya da iki raviyi birleştirmesi ise hiçbir kaynaktan türetilemez; kimin, ne zaman karar verdiğiyle birlikte bir kez kaydedilmelidir. Sistem bu yüzden her olgu için kararların izini tutar.

Bu ilke, Bölüm 2.3'teki ikinci sınıfı da açıklar. Modelin sessizce normalize ettiği bir değer, ne metinden doğrudan türetilmiş bir değerdir ne de kaydedilmiş bir insan kararıdır. Kabul edilmeden önce adı kayıtlı bir kural ya da bir insan tarafından doğrulanmalıdır.

3 İlgili çalışmalar

Bu bölümde hadis verileri, ravilerin çıkarımı ve kimliklerinin belirlenmesi, hadis bilgi grafları ve büyük dil modelleriyle (BDM) kurulan bilgi graflarının değerlendirilmesi üzerine çalışmaları inceliyoruz. Her çalışmada referansı kimin ürettiğini, kalitenin hangi düzeyde ölçüldüğünü ve etiketleyici uyumunun raporlanıp raporlanmadığını soruyoruz.

Çalışmaları Ekim 2026'da web araması yaparak ve birincil kaynakları okuyarak bulduk. Tarama sistematik değildi. Üç durum dışında tam metinleri okuduk: Oumer'in tezine açık erişim olmadığı için sayıları yazarın model deposundan aldık; Gao ve arkadaşlarının çalışmasında Bölüm 1 ile 7.1 arasını, Raja'nın çalışmasında Bölüm 6 ile 8 arasını okuduk. 2023 tarihli SemanticHadith makalesi ve Bölüm 3.5'te adı geçen graf kurma çerçeveleri için bibliyografik kayıtlara dayanıyor, sayı raporlamıyoruz.

3.1 Hadis derlemleri ve etiketli kaynaklar

Sanadset 650K, 650.986 kayıt içerir ve 926 kitabı kapsadığı belirtilir [5]. İndirdiğimiz dosyada 956 farklı kitap değeri vardır [6]. Metin web kazımayla toplanmış, bileşenleri ayrılmış ve raviler "düzenli ifadeler ve metin eşleştirme yardımıyla" etiketlenmiştir. Veri makalesi, doğruluk ölçümü ya da insan kontrolü raporlamaz. Bu etiketlere karşı elde edilen skor, kurallarla uyumu ölçer. Lisans, Hugging Face kopyasının veri kümesi kartında cc0-1.0 [29], ilk yüklemenin Mendeley Data kaydında ise "CC BY 4.0" olarak belirtilmiştir; her ikisini de 12 Ekim 2026'da okuduk.

Ifta Sunnah veri ailesi, derleyicisinin Ifta' Sunnah dijital kütüphanesinden web taraması ve ayrıştırma yoluyla çıkardığı tek bir derlemin Kaggle'daki üç sürümünden oluşur [6]. Özgün derlemde 33 kitap, 276.347 hadis ve 20.957 ravi profili vardır. Varlık ve ilişki sürümü, ravi metin aralıklarını ve aktarım ilişkilerini ekler; 223.478 hadisten oluşan sabit bir bölünme kullanır (156.434 eğitim, 33.521 doğrulama, 33.523 test). Graf sürümünde, her zincir adımının bir ravi kimliğine bağlandığı 257.840 hadis vardır. Etiketler, platformun üçüncü bir tarafça yeniden yapılandırılmış işaretlemeleridir. Kaynak, platformun bunları nasıl ürettiğini söylemez ve etiketleyici uyumu raporlamaz. 12 Ekim 2026'da Kaggle lisans alanında özgün derlem için "CC BY-SA 4.0", varlık sürümü için "Unknown", graf sürümü için "CC BY-NC-SA 3.0 IGO" yazıyordu. Belgeler, kullanıcılardan platformun koşullarına uymalarını ister.

AR-Sanad 280K, eğitim ve doğrulama için 279.625 yapay zincir ile altı temel hadis kitabından alınmış 27.056 gerçek zincirlik bir test kümesi sunar. Test kümesi 6.378 raviyi kapsar ve her ad bir ravi kimliği taşır [7]. Makaleye göre kaynak web sitesindeki ravi kimliklerini 30 uzmandan oluşan bir grup belirlemiştir. Test kümesi ayrılmış adları içerir, hadis metnini içermez; dolayısıyla kimliği sınayabilir, çıkarımı sınayamaz. Depoda lisans dosyası yoktur. Makalenin kaydında CC BY 4.0 lisans bağlantısı vardır; depo, bu lisansın veriyi de kapsayıp kapsamadığını söylemez [6].

3.2 İsnad ve ravi çıkarımı

Altammami ve arkadaşları, n-gram modelleriyle zinciri rivayet metninden ayırır [30]. Yazarlar, altı kitabın her birinden 40'ar hadis olmak üzere 240 hadisi etiketlemiş ve ayrım noktasının doğruluğunu hadis başına ölçmüştür. İkili gram modeli 240 hadisin 222'sinde (%92,5), üçlü gram modeli ise %48'inde doğru sonuç vermiştir. Raporlanan hata türleri arasında paralel zincirler vardır.

Muther ve Smith, uzun ve noktalama işareti kullanılmamış klasik metinlerde zincir metin aralıklarını saptar [31]. Derlemlerinde, model önerilerini düzelten İslam tarihi uzmanlarının etiketlediği 50 metin, 907.110 belirteç ve 3.071 zincir vardır. Uyum, 2.000 satır üzerinde 5 etiketleyiciyle ölçülmüştür: Krippendorff alfa değeri 0,84, ikili Cohen kappa değerleri ise 0,71 ile 0,96 arasındadır. Etiketleyiciler zincirin başlangıcında

vakaların %70'inde, bitişinde %53'ünde tam olarak uzlaşmıştır. Koşullu rastgele alan modeli, katı metin aralığı eşleştirmesinde F1 0,254'e, kısmi eşleştirmede 0,835'e ulaşmıştır. Bizim için iki nokta öne çıkar: eşleştirme ölçütü skoru büyük ölçüde değiştirir ve uzmanlar bile zincirin nerede bittiğinde uzlaşamaz.

Oumer, CAMeLBERT-CA, AraT5 ve AraBART modellerine Ifta varlık ve ilişki verileri üzerinde ince ayar yapar [1]. Yazarın model deposu, 33.523 test hadisinde CAMeLBERT-CA için ravi F1 değerini 0,98, ilişki çifti F1 değerini 0,95 olarak bildirir. AraT5 için ravi F1 değeri 0,97, çift F1 değeri 0,95, zincir numarasını içeren üçlülerin F1 değeri 0,93'tür. AraBART için üçlü F1 değeri 0,48'dir. Çiftler ve üçlüler, platform işaretlemesinden türetilen ilişkilerle tam eşleşmeye göre puanlanır. Veri bölünmesinde kitapların birbirinden ayrılıp ayrılmadığı belirtilmemiştir. Bu veri üzerinde model tabanlı çıkarımı ölçen bir çalışma bulamadık.

3.3 Ravi kimlik çözümleme

Mahmoud ve arkadaşları, kimlik belirlemeyi her ad geçişini 18.298 raviden birine sınıflandırma olarak ele alır [7]. Yapay doğrulama zincirlerinde model, 0 ile 100 arasındaki ölçekte 92,9 mikro-F1'e ve %30,2 zincir hata oranına ulaşır. Zincir hata oranı, en az bir ravisinin kimliği yanlış belirlenen zincirlerin yüzdesidir. Gerçek test zincirlerinde mikro-F1 83,5, makro-F1 78,8, zincir hata oranı ise %60,6'dır. Bu oran kitaplara göre %48,7 (al-Tirmidhi) ile %70,5 (al-Nasai) arasında değişir. Doğru ravi, vakaların %95,3'ünde ilk üç aday, %97,1'inde ilk beş aday içindedir. Doğrulama kümesinde Muhammad ve Abdullah gibi yaygın adların 3.669 geçişinin %26'sının kimliği doğru belirlenmiştir. Böylece geçiş düzeyindeki skor, zincir düzeyindeki başarısızlığı gizler; sıralanmış kısa bir liste, ilk yanıta göre doğru raviyi daha sık içerir.

Wirahman, etiketli bir referans kullanmadan Sanadset'teki 185.216 ravi adı varyantını biyografik veritabanlarına bağlar [4]. Varyantların 94.628'i (%51,1) bağlanmış, 88.343'ü (%47,7) için aday bulunamamıştır. Doğruluk ölçülmemiş, eşikler gözle incelemeyle seçilmiştir. Yazar, uzmanların onayladığı 1.000 ile 2.000 bağlantıyı gelecek çalışmaya bırakır. Oluşan grafın derecesi en yüksek iki düğümü "babası" (7.612) ve "babam" (5.550) olmuştur. Bu, adı verilmeyen ravileri metindeki dizgilerine göre birleştirmenin etkisini gösterir.

3.4 Hadis bilgi grafları ve model tabanlı yapılandırma

SemanticHadith, hadis derlemi için ontoloji güdümlü bir bilgi grafıdır [2]. 2026 tarihli genişletmesi, 2.0.1 sürümü, altı kitaptaki 34.458 hadisten 303.869 varlık, 6.822 kişi ve 47.496 benzerlik bağlantısı içeren bir graf kurar [3]. Metindeki varlıklar bir adlandırılmış varlık tanıyıcıyla çıkarılır (mikro-F1 %97, makro-F1 %85) ve ad sözlüğü aracılığıyla ontoloji örneklerine eşlenir. Eşleşmeyen varlıklar elenir. Ontolojide ravi zinciri ve segmentleri için sınıflar vardır. Graf için üç alan uzmanı her kitaptan rastgele 100 örneği kontrol etmiş, hiçbir uyuşmazlık raporlanmamıştır; uyum istatistiği verilmez. Çalışmada model kullanılmaz, olgu başına karar kaydedilmez ve kaynak alıntısına karşı kontrol yoktur. Aynı adı paylaşan kişiler elle inceleme için işaretlenir ya da dışarıda bırakılır. Belirsiz adları aday listesiyle bir uzmana sunan bir çerçeve, gelecek çalışma olarak belirtilir.

Rezwan, modellerle işlenmiş bir rivayet derlemidir [32]. Altı alan uzmanı, rastgele seçilen 1.213 rivayeti her boyut için 0 ile 10 arasında puanlamıştır; uyum istatistiği verilmez. Zincir ile rivayet metninin ayrılması 9,30 puan almıştır (özette 9,33). Hata oranı %60'ı aşan rivayetler, tüm rivayetlerin %5,85'i, ortalamadan çıkarılmıştır.

Çinici, Gemini 2.5 Flash ile bilgi getirme kullanarak klasik bir biyografi eserinin tek cildinden 271 âlim için 21 değişkenli bir biyografik veri kümesi kurar [33]. Yazar, ilk 100 kaydı özgün metinle karşılaştırmış ve sayı vermeden doğruluğun yüksek olduğunu belirtmiştir. Görev zincirler değil biyografidir; graf ya da onay adımı yoktur.

3.5 Model tabanlı bilgi grafı kurma ve değerlendirme

Extract, Define, Canonicalize, KGGen ve iText2KG gibi genel çerçeveler, modellerle metinden graf kurar [14, 34, 35].

Text2KGBench, ontoloji güdümlü çıkarım için üç ölçüt ailesi içeren bir değerlendirme ölçütüdür: referans üçlülere karşı precision, recall ve F1; ontolojiye uygunluk; özne ve nesnenin cümlede ya da ontolojide geçip geçmediğini sınayan bir halüsinasyon ölçüsü [16]. Wikidata-TekGen bölümünde Vicuna-13B, precision 0,38, recall 0,34, F1 0,35, ontolojiye uygunluk 0,83 ve özne halüsinasyonu 0,17 elde etmiştir.

Gao ve arkadaşları, tüm grafın doğruluğunu bir örneklemden tahmin eder [36]. MOVIE graflarında, 174 varlıktan alınan 174 üçlülük rastgele örneklem 3,53 saat sürerken 24 varlıktan alınan 178 üçlülük iki aşamalı küme örneklemesi 1,4 saat sürmüştür. Her ikisinde de hata payı yaklaşık %5'tir. Tahmin yansızdır ve güven aralığıyla birlikte verilir.

3.6 Alan derlemeleri

Haque ve Islam, 32 kaydı altı boyutta değerlendirir: derlemin gerçekçiliği, değerlendirme tasarımı, aktarım, yeniden üretilebilirlik, uzman katılımı ve akademik kullanım [9]. Değerlendirme tablolarındaki 8 satır arasında uzman katılımı yalnızca Rezwan için tam olarak puanlanmıştır. Altı kitapla sınırlı bir değerlendirme kültürü, sentetik ve gerçek veri arasında bir fark, karşılaştırılamayan ölçütler ve seyrek uzman doğrulaması saptarlar.

Mhamedi ve arkadaşları 57 çalışmayı derler [37]. Ravi çıkarımı tablolarında raporlanan F1 değerleri %33 ile %99,63 arasında değişir; test kümeleri 102 ile 200 hadis içerir ve doğruluk, precision ve F1 birbirine karıştırılır.

3.7 Çalışmanın konumu

Çalışma	Ne sağlar?	Kalite nasıl ölçüldü?	Amacımız açısından ne eksik?
Sanadset 650K [5]	Ravileri etiketlenmiş 650.986 kayıt	Ölçülmedi; etiketler düzenli ifadelerden gelir	Kontrol edilmiş referans; ravi kimliği
Ifta veri ailesi [6]	Ravi metin aralıkları, ilişkiler ve kimlikler	Uyum sayısı yok; işaretlemenin kökeni açıklanmıyor	Lisansın teyidi; model tabanlı bir çalışma
AR-Sanad 280K [7]	Kimlik içeren 27.056 gerçek zincir; sınıflandırıcı	Mikro ve makro F1, zincir hata oranı, ilk k aday	Hadis metni yok, dolayısıyla çıkarım testi yapılamaz
Altammami ve arkadaşları [30]	Zincir ile rivayet metninin bölütlenmesi	240 hadiste hadis başına doğruluk	Ravi düzeyinde çıktı
Muther ve Smith [31]	Uzman etiketli derlem; metin aralığı saptama	Katı ve kısmi metin aralığı F1'i; 5 etiketleyicinin uyumu	Ravi adları ve ilişkiler
Oumer [1]	İnce ayarlı ravi ve ilişki çıkarımı	33.523 test hadisinde tam eşleşme (yazarın model deposu)	Eğitim verisi olmayan bir düzen; karar kayıtları
Wirahman [4]	185.216 ad varyantının otomatik bağlanması	Ölçülmedi	Etiketli referans; insan onayı
SemanticHadith 2.0.1 [3]	34.458 hadisin ontolojisi ve grafı	Tanıyıcı F1'i; örneklerin uzman kontrolü	Olgu başına karar; kaynak alıntıları
Rezwan [32]	Modellerle işlenmiş derlem	0 ile 10 arasında uzman puanları	Sayıma dayalı precision ve recall
Çinici [33]	Modelle kurulmuş biyografik kayıtlar	Tek yazar, 100 kayıt, sayı yok	Ölçülmüş hata sınıfları; onay adımı
Text2KGBench [16]	Değerlendirme ölçütü ve üç ölçüt ailesi	Referans üçlüler, ontolojiye uygunluk, metinde bulunma	Değer-alıntıda testi; insan adımı
Gao ve arkadaşları [36]	Graf doğruluğu için örnekleme tasarımı	Güven aralıklı yansız tahmin	Burada uygulanmadı; etiketli örneklemin ötesindeki

Çalışma	Ne sağlar?	Kalite nasıl ölçüldü?	Amacımız açısından ne eksik?
			doğruluğu tahmin etmek için aday

Böyle bir projenin iddia edebileceği şeylerin çoğu zaten vardır. Altı kitabın grafı yayımlanmıştır ve ontolojisi zincir segmentlerini sınıf olarak modeller. Ravi bağlama için etiketli gerçek veri ve ölçülmüş bir taban çizgisi vardır. Yazarın model deposuna göre ince ayarlı modeller, platform etiketlerine karşı 0,95 ve üzeri F1 değerleriyle ravileri ve aktarım ilişkilerini çıkarır. Zincir metin aralıkları için uyum sayıları verilen uzman etiketli bir derlem vardır; metinde bulunma testi de yerleşik bir ölçüttür. Bu nedenle yeni bir şema, aktarım olayı için yeni bir gösterim ya da daha yüksek çıkarım doğruluğu iddia etmiyoruz.

İncelediğimiz çalışmalarda üç şey bulamadık: model ile graf arasındaki her doğrulama adımının katkısını ayrı ayrı ölçen bir çalışma; çıkarılan değerin modelin kendi alıntısında geçip geçmediğine bakan ve ayrı bir hata sınıfı olarak raporlanan bir kontrol; her olguyu kimin ya da neyin kabul ettiğini gösteren bir kayıt. Bu tezin konusu bunlardır. Sistem, model ile graf arasına deterministik kontroller ve bir insan inceleme kuyruğu koyar, her adımın neyi yakaladığını sayar ve her olgu için bir kabul kaydı saklar. Tarama sistematik olmadığı için "bulunamadı", "yoktur" anlamına gelmez.

4 Gereksinimler ve kapsam

Bu bölümde sistemin ne yapması gerektiğini, her gereksinimin nerede gerçekleştirildiğini ve hangi testle kontrol edildiğini belirtiyoruz. Başka bir dizinle başlamadıkları sürece gerçekleştirim yolları `backend/src/app/` dizinine göredir. Test dosyaları `backend/tests/unit/` ya da `backend/tests/integration/` dizinindedir. Bir gereksinimi ancak kodu `main` dalında bulunuyorsa listeliyoruz.

4.1 Kullanıcılar ve kullanım senaryoları

Sistemde kullanıcı hesabı yoktur; dolayısıyla aşağıdaki roller yazılım tarafından uygulanmaz.

- **Şema sahibi.** Proje oluşturur, düğüm ve ilişki tiplerini tanımlar ya da şema şablonu uygular ve bir şema sürümünü etkinleştirir.
- **Operatör.** Kaynak metni yükler, işlemeyi başlatır ve bir koşuyu grafa commit eder.
- **İnceleyici.** İnceleme kuyruğundaki adayları kabul eden, reddeden ya da düzenleyen, alan bilgisine sahip kişidir. İnceleyici bir ad yazar; bu ad her kararla birlikte saklanır.
- **Okuyucu.** Commit edilmiş grafı gezer; vaka çalışmasında bir hadisi ravi zincirleri, alıntıları ve kabul kayıtlarıyla birlikte okur.

Kullanım senaryoları şunlardır: şemalı bir proje kurmak, metinleri yüklemek, bir metni işleyerek doğrulanmış adaylar üretmek, açık inceleme öğeleri hakkında karar vermek, bir koşuyu commit etmek ve commit edilmiş bir olgunun kaynak metnine ve kabulüne kadar izini sürmek.

4.2 İşlevsel gereksinimler

Kimlik	Gereksinim	Gerçekleştirildiği yer	Doğrulayan test
FR1	Projenin düğüm tipleri, ilişki tipleri, özellikleri ve birleştirme anahtarları içeren sürümlenmiş şemaları vardır. Adlar ASCII kimlikleridir. Yalnızca taslak sürüm düzenlenebilir.	`application/use_cases/schema_management.py`, `domain/schema/identifiers.py`	`test_project_schema_api.py`
FR2	Proje oluşturulurken aynı istekte bir şema şablonu uygulanıp etkinleştirilebilir.	`ProjectSetupService`, `infrastructure/template/catalog.py`	`test_template_project_api.py`
FR3	Kaynak metin tek tek birimler hâlinde ya da toplu yüklenir. Yinelenen dış referans her öğe için raporlanır.	`IngestionManagementService`	`test_ingestion_api.py`, `test_bulk_ingest_api.py`

Kimlik	Gereksinim	Gerçekleştirildiği yer	Doğrulayan test
FR4	Kaynak birimi, adı belirtilen bir stratejiyle segmentlere ayrılır: `paragraphs` ya da ravi zinciri başına bir segment ve kaynak konumları üreten `isnad`.	`domain/extraction/segmentation.py`, `domain/hadith/segmentation.py`	`test_hadith_segmenter.py`, `test_isnad_segmentation.py`
FR5	Sağlayıcı, segment başına adaylar önerir. Başarısız bir çağrı diğer çağrıları iptal etmez. Tanımsız tip ya da bildirilmemiş özellik tek başına elenir ve şema önerisi olarak kaydedilir.	`ExtractionManagementService.extract_candidates`	`test_isnad_pipeline_api.py`, `test_grounding_extraction_api.py`
FR6	Adaylar şemaya göre doğrulanır: tip, zorunlu özellikler, birleştirme anahtarları, uç düğüm tipleri, izin verilen değerler, güven skoru eşikleri ve yinelenen adaylar kontrol edilir.	`PipelineManagementService.validate_run`	`test_pipeline_api.py`
FR7	Çıkarılmış adayın alıntısı segmentinde, düğümün birleştirme anahtarı değeri de alıntıda geçmelidir. Başarısız kontrol, adayı incelemeye gönderir.	`_evidence_findings`, `_merge_key_grounding_findings`, `domain/hadith/grounding.py`	`test_hadith_grounding.py`, `test_grounding_extraction_api.py`
FR8	Aynı iki düğüm arasındaki, birleştirme anahtarı değerleri farklı iki ilişki, iki kenar olarak saklanır.	`domain/shared/resolution.py`, `infrastructure/persistence/neo4j/repositories.py`	`test_resolution.py`, `test_commit_api.py`
FR9	İnceleyici bir inceleme öğesini	`ReviewManagementService.decide_review_item`, `validate_run`, `review_decisions` tablosu	`test_review_decisions_api.py`, `test_review_api.py`

Kimlik	Gereksinim	Gerçekleştirildiği yer	Doğrulayan test
	kabul eder, reddeder ya da düzenler. Her karar, inceleyici, zaman ve isteğe bağlı süreyle birlikte saklanır; kararlara yalnızca yeni kayıt eklenir. Otomatik adımlar, hakkında karar verilmiş adayı değiştirmez.		
FR10	Sistem, koşunun her doğrulanmış adayı için onu kabul eden insan kararını ya da otomatik politikayı raporlar.	`ReviewManagementService.list_acceptance_records`	`test_review_decisions_api.py`
FR11	Koşu bir kez commit edilir; bunun için bekleyen aday ve açık inceleme öğesi bulunmamalıdır.	`CommitManagementService.commit_run`	`test_commit_api.py`
FR12	Çıkarım, doğrulama ve commit, bir işçinin tükettiği kuyruklanmış işler olarak ya da kaynak birimleri listesi için tek bir eşzamanlı istekte çalışır.	`processing_job_management.py`, `interfaces/cli/processing_jobs_worker.py`, `SourceUnitProcessingService`	`test_processing_jobs_api.py`, `test_bulk_ingest_api.py`
FR13	Commit edilmiş graf okunabilir; commit edilmiş bir hadis de alıntılar ve kabul kayıtları içeren sıralı zincirler olarak okunabilir.	`application/use_cases/exploration_management.py`	`test_exploration_api.py`, `test_chains_api.py`
FR14	Tarayıcı paneli yukarıdaki adımları yürütür.	`frontend/src/modules/projects/`	Otomatik test yoktur; CI derler

4.3 İşlevsel olmayan gereksinimler

Kimlik	Gereksinim	Gerçekleştirildiği yer	Doğrulayan test
NFR1	İzlenebilirlik: commit	`DERIVED_FROM`, `SUPPORTED_BY` ve `RelationEvidence`, `Neo4jCanonicalGraphRepository` tarafından yazılır	`test_exploration_api.py` (bellekte), `test_realdb_isnad_flow.py` (`real_db`)

Kimlik	Gereksinim	Gerçekleştirildiği yer	Doğrulayan test
	edilmiş düğüm ya da kenar, kaynak birimine ve onu destekleyen segmentlere bağlanır.		
NFR2	Atomik graf yazımı: başarısız commit, grafta koşunun hiçbir parçasını bırakmaz.	`commit_bundle`, `_write_bundle` işlemini tek bir `execute_write` işlemi içinde çalıştırır	`test_realdb_isnad_flow.py` (`real_db`)
NFR3	Motor, veritabanı, ağ ya da API anahtarı olmadan çalışır ve test edilir.	`infrastructure/persistence/memory/`, `infrastructure/llm/rule_based_isnad_provider.py`	Varsayılan test koşusu, `test_isnad_baseline_script.py`
NFR4	Alana özgü davranış, bölütleme stratejisi, şablon ya da sağlayıcı olarak eklenir.	`SegmentationStrategy`, `StaticDomainTemplateCatalog`, `StaticLLMProviderRegistry`, `bootstrap/container.py`	`test_isnad_pipeline_api.py`, `test_rule_based_isnad_provider.py`
NFR5	Yinelenen model çağrıları önbellekten karşılanır ve belirteç kullanımı çağrı başına raporlanır.	`infrastructure/llm/cached_provider.py`, `infrastructure/llm/anthropic_provider.py`	`test_llm_cache_provider.py`, `test_anthropic_provider.py`
NFR6	Veritabanı şeması yalnızca göçler yoluyla oluşturulur ve ORM modelleriyle eşleşir.	`backend/migrations/versions/`, `0001` ile `0006` arası	`test_review_decisions_migration.py`, `test_realdb_migrations.py` (`real_db`)
NFR7	Arka uç kodu her push işleminde lint ve katı tip kontrolünden geçer.	`backend/pyproject.toml`, `.github/workflows/ci.yml`	CI iş akışı

`real_db` işaretli testler çalışan veritabanları gerektirir; aksi durumda atlanır (Bölüm 6.7). NFR4, Bölüm 5.1'de listelenen istisnalarla geçerlidir.

4.4 Kapsam dışı

Her öğeyi koda karşı kontrol ettik.

- **Ravi kimlik çözümleme.** `Narrator` tipinin tek birleştirme anahtarı `name`'dir. Karşılaştırmada yalnızca Unicode NFKC normalizasyonu, ardışık boşlukların tek boşluğa indirgenmesi ve büyük-küçük harf katlama uygulanır (`normalize_text`). Aynı yazılı adı taşıyan kişiler bir düğümü paylaşır; iki farklı biçimde yazılan tek kişi ise iki düğüm olur.
- **Biyografik tarihler gerektiren alan kuralları.** İsnad şemasında tarih özellikleri yoktur; doğrulamada da iki ravinin aynı dönemde yaşayıp yaşamadığını kontrol etmek gibi kurallar için bir kanca bulunmaz.
- **Sıhhat derecelendirmesi.** Hiçbir tip ya da özellik, hadis veya ravi için bir derece tutmaz.
- **Kimlik doğrulama ve kullanıcılar.** Hiçbir yönlendiricinin kimlik doğrulama bağımlılığı yoktur. İnceleyici adı serbest metindir.
- **Commit edilmiş koşunun geri alınması.** API'de 33 `GET` ve 20 `POST` uç noktası vardır; `PUT`, `PATCH` ya da `DELETE` yoktur. Tamamlanmış koşu değiştirilemez.
- **Türkçe hadis metni.** Normalizasyon ve aktarım kalıpları sözlüğü yalnızca Arap yazısını kapsar. Yalnızca panel arayüzü Türkçedir.
- **Graf üzerinden soru yanıtlama.** Okuma uç noktaları düğümleri, ilişkileri ve zincirleri listeler ve gösterir. `EmbeddingProvider` protokolü, `application/ports/llm_provider.py` içinde bildirilmiştir; ancak gerçekleştirilmemiştir ve bilgi getirme uç noktası yoktur.

İki şema alanı saklanır, ancak uygulanmaz. Özelliğin `data_type` alanı modele gönderilir ve doğrulama sırasında kontrol edilmez. İlişki tipinin `directional` bayrağı ise saklandıktan sonra kullanılmaz.

5 Sistem tasarımı

Bu bölümde `main` dalında gerçekleştirilen tasarımı anlatıyoruz. Yollar `backend/src/app/` dizinine göredir. Bazı kararlar, önceki kodun incelemesinde bulunan kusurları giderir (tasarım analizi, 30 Eylül 2026). Tasarımı açıklamaya yardımcı olduğunda ilgili kusuru belirtiyoruz.

5.1 Tasarım ilkeleri

Model önerir ve grafa yazamaz. Sağlayıcı, önerilen düğüm ve ilişkileri döndürür; bunlar `CandidateNode` ve `CandidateRelation` satırları olarak `pending` durumunda saklanır. Tek graf yazma işlemi olan `CanonicalGraphRepository.commit_bundle`, yalnızca `CommitManagementService.commit_run` tarafından çağrılır ve yalnızca `validated` adayları alır.

Her olgu bir alıntı taşır. Aday, `EvidenceRef` değerleri tutar: segment kimliği, alıntı ve isteğe bağlı konumlar. Kanıtı olmayan çıkarılmış aday `missing_evidence` olarak işaretlenir. Bu bir kontroldür, saklama kısıtı değildir: inceleyici böyle bir adayı yine de kabul edebilir; elle girilen adaylar ise kontrol edilmez.

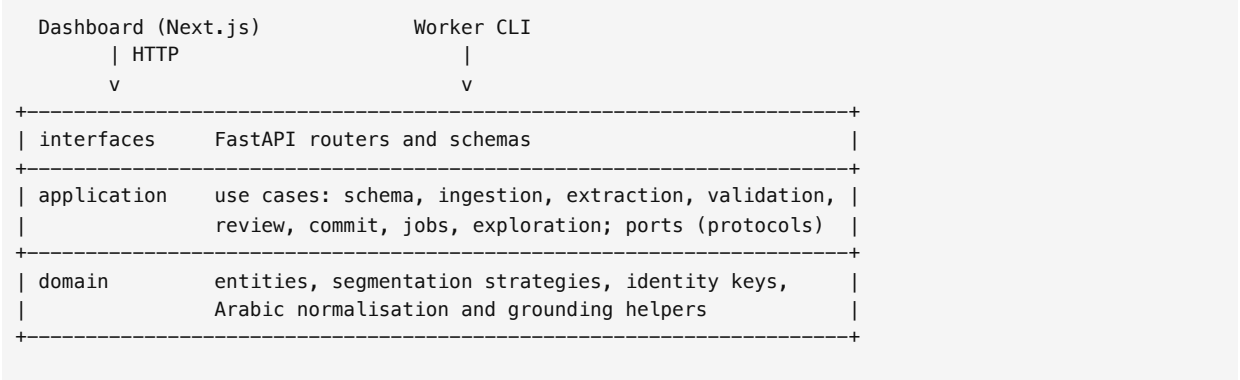
Kontroller sessizce reddetmek yerine inceleme için işaretler. Başarısız kaynağa dayandırma, değer, eşik ya da yinelenen aday kontrolü, durumu `review` yapar ve gerekçe kodu içeren bir inceleme öğesi oluşturur. Adayı kullanılamaz bırakan yapısal hatalar durumu `rejected` yapar; ancak her biri doğrulama sonucu olarak saklanır.

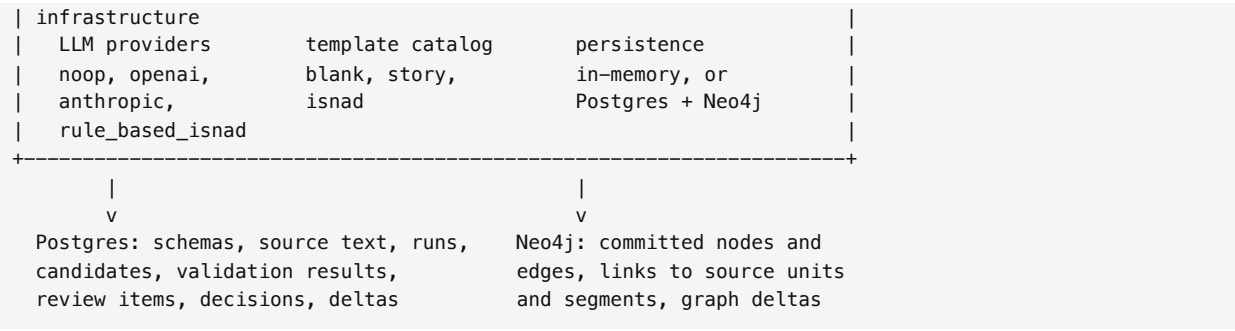
Her kabul, insan kararı ya da adı kayıtlı bir politika olarak kaydedilir. Doğrulanmış adayın izi, `review_decisions` içindeki bir satıra ya da adayı geçiren kuralla birlikte `auto_validated` politikasına kadar sürülebilir.

Alana özgü davranış eklentilerle eklenir; motor kodu genel kalır. Bölütleme bir `SegmentationStrategy` ile yapılır, şema bir şablondan gelir ve çıkarım bir `LLMProvider` üzerinden yürür. Çıkarım, doğrulama, inceleme ve commit kullanım senaryoları ravi ya da hadis adları içermez. Üç istisna vardır. Doğrulama, metin karşılaştırmasını `domain/hadith/grounding.py` içinden alır; bu yüzden Arapça katlama her projeye uygulanır. Zincir okuma modeli (`domain/exploration/chains.py`, `ChainExplorationService`, `/chains` uç noktaları), `Narrator` ve `NARRATED_FROM` tiplerini adlarıyla kullanır. Eklentiler, `bootstrap/container.py` içinde kodla kaydedilir.

5.2 Mimari

Arka uç, dört katmanlı tek bir FastAPI uygulamasıdır. Uygulama ve alan katmanları, altyapı ya da arayüz katmanlarından hiçbir şey içe aktarmaz: kullanım senaryoları `application/ports/` içindeki protokollere bağımlıdır, altyapı katmanı ise bunları gerçekleştirir.





Postgres, yapılandırmayı, kaynak metni, henüz kabul edilmemiş her şeyi ve denetim izini tutar. Neo4j, yalnızca commit edilmiş öğeleri, kaynak birimlerine ve segmentlere geri bağlantılarıyla birlikte tutar. Depodaki ADR-002 bunun gerekçesini verir: üst veri yapılandırılmış kayıtlardan oluşur; commit edilmiş graf ise yoğun bağlantılıdır ve dolaşılarak okunur. Bu ayrımda aday hiçbir zaman graf deposunda bulunmaz. Bedeli, iki deponun tek bir işlemde güncellenmemesidir (Bölüm 6.9).

Bellekte çalışan arka uç, aynı depo protokollerini sözlüklerle gerçekleştirir. `REPOSITORY_BACKEND`, `postgres` olmadığında varsayılan olarak bu arka uç kullanılır; test paketi de onu kullanır.

5.3 Veri modeli

Varlık	Ana alanlar	Saklandığı yer
Proje	ad, `template_id`, durum	`projects`
Şema sürümü	`version_no`, durum (`draft`, `published`, `active`, `retired`)	`schema_versions`
Düğüm tipi	ad, `display_name`, `merge_keys`, `confidence_threshold` (varsayılan 0,5), `extraction_hints`	`node_types`
İlişki tipi	ad, `from_type`, `to_type`, `merge_keys`, `confidence_threshold` (varsayılan 0,7)	`relation_types`
Özellik	ad, `data_type`, zorunlu, `enum_values`	`property_definitions`
Kaynak birimi	`raw_content`, `external_ref`, durum	`source_units`
Segment	sıra, metin, üst veri (strateji, zincir numarası, kaynak konumları)	`segments`
Koşu	kaynak birimi, şema sürümü, sağlayıcı, model, durum	`processing_runs`
Aday düğüm, aday ilişki	tip, özellikler, güven skoru, kanıt referansları, durum; düğümde `merge_key_payload`, ilişkide iki aday düğüm referansı bulunur	`candidate_nodes`, `candidate_relations`
Doğrulama sonucu	`candidate_ref`, `rule_code`, ciddiyet, sonuç, mesaj	`validation_results`
İnceleme öğesi	`item_type`, `item_ref`, `reason_code`, veri yükü, durum	`review_items`
İnceleme kararı	Bölüm 5.7'ye bakınız	`review_decisions`
Graf farkı	bir koşuda oluşturulan, birleştirilen ve reddedilen öğe sayıları	`graph_deltas`, `GraphDelta` düğümü
Kanonik düğüm	etiketleri `CanonicalNode` ve tip adıdır; anahtarı `project_id`, `node_type`, `merge_key` alanlarıdır	Neo4j
Kanonik kenar	bağlantı ilişki tipinin adını taşır; anahtarı `project_id`, `relation_type`, `relation_identity_key` alanlarıdır	Neo4j

Kanıt referansları, aday satırlarında JSON olarak saklanır. Yalnızca taslak şema sürümü düzenlenebildiği için bir koşunun tipleri değişmez.

5.4 Adım adım boru hattı

Metninde ح zincir geçiş işareti bulunan `Muslim:978` referanslı bir hadisi izliyoruz. Bölütleyici bu hadis için iki zincir döndürür. Referans etiketleri üç zincir verir: üçüncüsü rivayet metni başladıktan sonra başlar ve bölütleyici onu döndürmez.

1. **Yükleme.** Metin, `external_ref = "Muslim:978"` değerini taşıyan ve projenin etkin şema sürümüne bağlı bir kaynak birimi olur.
2. **Koşuyu başlatma.** `ProcessingRun`, şema sürümünü, sağlayıcıyı ve modeli kaydeder. Kaynak birimi `processing` olarak işaretlenir; bu durumda ikinci bir koşu başlatılamaz.
3. **Bölütleme.** `isnad[ stratejisi, bulduğu her zincir için bir segment döndürür. Yeni zinciri yalnızca ح işaretinde başlatır; bu yüzden çoğu hadis tek segment verir. Her segmentin metni, ]{lang=ar dir=rtl}raw_content[start:end]` alt dizgisinin aynısıdır; konumlar segment üst verisinde saklanır. Zincir bulunamazsa metnin tamamı, `fallback: whole_text` işaretli tek segment olur.
4. **Çıkarım.** `isnad` stratejisinde motor segment başına bir sağlayıcı çağrısı yapar; `paragraphs` stratejisinde ise kaynak biriminin segmentlerini tek çağrıda gönderir. Veri yükü, kaynak birimi referansını, ipuçlarıyla birlikte şemayı ve segmenti içerir. Yanıt, `StructuredExtractionContract` ile ayrıştırılır. Tanımsız tip taşıyan düğüm ya da ilişki elenir ve şema önerisi olarak kaydedilir; bildirilmemiş özellik de aynı biçimde kaldırılır. Çağrının dışındaki bir segmente atıf yapan, tanımsız düğüm referansı kullanan ya da yanlış tipteki düğümleri bağlayan yanıt, o çağrıyı başarısız kılar. Farklı çağrılardan gelen, tipi ve birleştirme anahtarı değeri aynı olan düğümler, tüm alıntıları koruyan tek bir aday olur. Koşu ancak bütün çağrılar başarısız olursa başarısız olur.
5. **Doğrulama.** `validate_run`, Bölüm 5.5'teki kontrolleri uygular, her adayı `validated`, `review` ya da `rejected` durumuna getirir ve inceleme öğelerini oluşturur.
6. **İnceleme.** İnceleyici, açık öğeler hakkında karar verir (Bölüm 5.7). Her karardan sonra bir düğümü bekleyen ilişkiler yenilenir.
7. **Commit.** `commit_run`, hâlihazırda graf farkı, bekleyen adayı, açık inceleme öğesi, uç düğümü doğrulanmamış doğrulanmış ilişkisi, birleştirme anahtarı değeri olmayan doğrulanmış ilişkisi ya da farklı özelliklerle tek kenara dönüşecek iki doğrulanmış ilişkisi bulunan koşuyu reddeder. Aksi durumda doğrulanmış adaylar tek bir graf işleminde yazılır, fark saklanır ve koşu `completed` durumuna geçer.
8. **Okuma.** Zincir uç noktaları, hadisi sıralı zincirleriyle ve her bağlantının alıntısı ve kabul kaydıyla birlikte döndürür.

2 ile 5 arasındaki adımlar ve açık öğesi olmayan koşular için 7. adım, tek istekte çalışabilir (`POST /api/projects/{project_id}/source-units/process`). 4, 5 ve 7. adımlar kuyruklanmış işler olarak da çalışabilir.

5.5 Doğrulama kontrolleri

Gerekçe kodu	Tetikleyen durum	Etki	Gerekçe
`node_type_exists`, `relation_type_exists`	Tip şema sürümünde yoktur	Reddedilir	Graf etiketleri yalnızca şemadan gelir; bu duruma yalnızca elle girilen kayıtlarda ulaşılır
`required_node_property`, `required_relation_property`, `node_merge_key`	Zorunlu özellik ya da düğümün birleştirme anahtarı değeri eksiktir	Reddedilir	Şemaya göre eksiktir; kimliği olmayan düğüm birleştirilemez
`relation_candidate_refs`, `relation_from_type`, `relation_to_type`	Uç düğüm koşuda yoktur ya da düğüm tipi ilişki tipinden farklıdır	Reddedilir	Şemaya uygunluk

Gerekçe kodu	Tetikleyen durum	Etki	Gerekçe
`relation_dependency_rejected`	Uç düğüm reddedilmiştir	Reddedilir	Kenar her iki düğüme de ihtiyaç duyar
`missing_evidence`	Çıkarılmış adayın kanıt referansı yoktur	İncelemeye gider	Alıntısı olmayan olgu kontrol edilemez
`quote_not_in_segment`	Alıntı boştur ya da atıf yapılan segmentte bulunamaz	İncelemeye gider	Alıntı uydurulmuş olabilir
`value_not_in_quote`	Düğümün dizgi türündeki birleştirme anahtarı değeri hiçbir alıntısında geçmez ve kaynak biriminin `external_ref` değerinden farklıdır	İncelemeye gider	Ad değiştirilmiş ya da uydurulmuş olabilir
`property_value_not_allowed`	Dizgi değeri, özelliğin `enum_values` listesinin dışındadır	İncelemeye gider	Değer sapması
`node_confidence_threshold`, `relation_confidence_threshold`	Güven skoru tipin eşiğinin altındadır	İncelemeye gider	Model kuşku bildirir
`missing_relation_merge_key`	İlişkide bildirilmiş bir birleştirme anahtarının değeri eksiktir	İncelemeye gider	İnceleyici değeri sağlayabilir
`relation_dependency_review`	Uç düğüm incelemededir	İncelemeye gider	Kenar düğümünü bekler
`potential_duplicate_node`, `potential_duplicate_relation`	Koşunun başka bir adayı aynı kimliği taşır	Biri dışında hepsi incelemeye gider	Tek bir graf öğesine dönüşürler
`schema_type_proposal`, `schema_property_proposal`	Çıkarım tanımsız tip ya da bildirilmemiş özellik üretmiştir	Aday elenir ya da özellik kaldırılır; `schema_proposal` tipinde inceleme öğesi oluşturulur	Sapma kaydedilir, çağrı devam eder
`candidate_node_valid`, `candidate_relation_valid`	Bütün kontroller geçilmiştir	Doğrulanır	`auto_validated` politikasının dayanağıdır
`canonical_node_match`, `canonical_relation_match`	Aynı kimlik zaten grafta vardır	Durum değişmez	İnceleyiciye bağlam sağlar
`human_review_decision`	Adayın kayıtlı kararı vardır	Durum karara göre belirlenir	Bölüm 5.7

Alıntılar ve değerler, `normalize_for_grounding` uygulandıktan sonra karşılaştırılır. Bu işlev önce motorun `normalize_text` işlevini (Unicode NFKC, ardışık boşlukların tek boşluğa indirgenmesi, büyük-küçük harf katlama), ardından `normalize_arabic` işlevini uygular. İkincisi, kısa ünlü ve Kur'an işaretlerini ve uzatma karakterini kaldırır, noktalama işaretlerini boşluklarla değiştirir ve elif biçimlerini ى, ا harfini ة, ي harfini ؤ, ه harfini و ve ئ harfini ي biçimine katlar. Karşılaştırma, alt dizgi içerme testidir; boş alıntı hiçbir zaman eşleşmez. İlişki özelliklerinin değerleri alıntıyla karşılaştırılmaz.

Başarısız kontrol, güven skoru ne olursa olsun her zaman incelemeye yönlendirir. Güven skoru, modelin kendisi hakkında bildirdiği bir sayıdır; bu yüzden şemaya uyan uydurma bir olgu yüksek değer taşıyabilir. Kontrolün sonucu buna bağlı değildir. Aday reddedilmez de; çünkü başarısızlığın geçerli bir nedeni olabilir: ada belirli tanımlık ekleyen model, `value_not_in_quote` kontrolünden geçemez ve bu durum hakkında bir insan karar vermelidir. Bağımlılık yenileme yalnızca bağımlılık gerekçeli öğeleri kapattığı için bu öğeleri açık bırakır.

Kanıt ve değer kontrolleri, yalnızca koşuda başarılı bir çıkarım yürütümü bulunduğunda çalışır; API üzerinden girilen adaylar operatörün yazdığı adaylar olarak ele alınır.

5.6 İlişki kimliği

Önceki kodda kenar, iki uç düğümü ve tipi üzerinden birleştiriliyor, ardından özelliklerinin üzerine yazılıyordu. İki hadiste aynı iki ravi arasındaki iki aktarım tek kenar oluyor; ikinci aktarım, ilkinin hadis referansını ve aktarım ifadesini değiştiriyordu. Adaylar arasındaki yinelenme kontrolü, anahtar olarak tüm özellikleri kullanıyordu; bu yüzden iki farklı aday görüyor ve inceleme öğesi oluşturmuyordu. Kayıp sessizdi.

Artık ilişki tipi `merge_keys` bildirir; bunlar, tipte bildirilmiş özelliklerin adları olmalıdır. `build_relation_identity_key`, normalize edilmiş birleştirme anahtarı değerlerini SHA-256 ile özetler; Neo4j ifadesi bu anahtar üzerinden birleştirir:

```
MERGE (from_node)-[r:`NARRATED_FROM` {project_id: $project_id,
       relation_type: $relation_type,
       relation_identity_key: $relation_identity_key}]->(to_node)
```

Aday düzeyindeki saptama aynı kimliği kullanır. `build_candidate_relation_resolution_key`, ilişki tipini, her iki uç düğümün kimlik anahtarlarını ve ilişki kimlik anahtarını bir araya getirir; birleştirme anahtarları dışındaki özellikleri dikkate almaz. Dolayısıyla iki aday, tam olarak aynı kenara dönüşecekleri durumda yinelenmiş sayılır.

Birleştirme anahtarları olmayan tip, düğüm çifti başına tek kenar tutar. Ancak o çift için ikinci bir aday artık incelemeye gider; commit işlemi de farklı özelliklerle tek kenara dönüşen iki doğrulanmış ilişkiyi reddeder.

5.7 Karar kaydı

Kararlar `review_decisions` içinde saklanır. Depo protokolü yalnızca `add_decision` ve `list_decisions_by_run` sunar. Yeniden doğrulama, koşunun inceleme öğelerini değiştirir, kararlarını değiştirmez.

Karar; inceleme öğesini, koşuyu, öğe tipini ve öğe referansını, eylemi (`accept`, `reject`, `edit`), `reviewer_id` alanını, isteğe bağlı notu, isteğe bağlı `duration_ms` alanını, zamanı ve düzenleme yapıldıysa adayın önceki ve sonraki hâlini saklar. Düzenleme, adayın bir kopyasına uygulanır ve şemaya ve koşudaki yinelenen adaylara karşı kontrol edilir; reddedilen karar hiçbir şey saklamaz. Aynı öğe hakkındaki ikinci karar yeni kayıt olarak eklenir ve son karar geçerlidir. Tamamlanmış koşu karar kabul etmez.

Otomatik adımlar için insan kararları kesindir. `validate_run` önce kararları okur: hakkında karar verilmiş aday, durumunu son karardan alır, `human_review_decision` sonucunu alır, kontrolleri atlar ve inceleme öğesini korur. Yinelenen adayların ele alınması, hakkında karar verilmiş adayı hiçbir zaman yeniden incelemeye taşımaz; bağımlılık yenileme de hakkında karar verilmiş ilişkileri atlar.

Kabul kayıtları, istek geldiğinde `list_acceptance_records` tarafından hesaplanır; saklanmaz. Her doğrulanmış aday için kayıt, kabul eden son kararı ya da adayı geçiren kural koduyla birlikte `auto_validated` politikasını içerir.

İki sınır vardır: inceleyici adının kimliği doğrulanmaz ve karar tek bir koşunun aday kimliğine bağlıdır; bu yüzden aynı metin üzerinde yeni bir koşu bu kararı yeniden kullanmaz.

5.8 İsnad vaka çalışmasının eklentileri

Bölütleme stratejisi. `IsnadSegmentation`, kural tabanlı `segment_isnad[ işlevini sarmalar. Kaynak konumlarına eşleme tutarak metni normalize eder, ح işaretinde böler, 26 maddelik bir sözlükten aktarım kalıplarını bulur ve rivayet metninin ilk ipucunda zinciri bitirir. Strateji, ]{lang=ar dir=rtl}one_call_per_segment` değerini ayarlar.

Şema şablonu. `isnad` şablonu; `Narrator` (birleştirme anahtarı `name`), `Hadith` (birleştirme anahtarı `hadith_ref`), raviden raviye `NARRATED_FROM` (birleştirme anahtarları `hadith_ref`, `chain_no`, `position`; aktarım ifadesi için `sigha`

özelliği) ve raviden hadise `TRANSMITS` tiplerini bildirir. Bölütleme stratejisinin adını da belirtir. Proje oluşturma, `apply_template_schema` ayarlandığında bu şablonu uygular.

Kural tabanlı sağlayıcı. `rule_based_isnad`, dil modeli olmayan koşuldur. İstemi dikkate almaz, her segment üzerinde aynı bölütleyiciyi çalıştırır ve yazıldığı biçimde ad taşıyan ravi düğümleri ile ardışık her çift için bir `NARRATED_FROM` ilişkisi döndürür. `Hadith` düğümleri oluşturmaz. Alıntıları segmentin dilimleridir; dolayısıyla çıktısı, tasarımı gereği kaynağa dayandırma kontrollerinden geçer.

Çıkarım ipuçları. Her tip, veri yükünün şema kısmında modele iletilen ipucu dizgilerinden oluşan bir liste taşır. Örneğin "Ravi adını segment metninden birebir kopyala."

5.9 Tasarım kararları ve alternatifler

Karar	Değerlendirilen alternatif	Gerekçe
Kenar kimliğinin ilişki birleştirme anahtarlarından elde edilmesi	Her aktarım olayı için bir düğüm	Genel modelde daha küçük bir değişiklik gerektirir
Başarısız kontrolün incelemeye yönlendirmesi	Reddetmek ya da güven skoru eşiğinin üzerinde kabul etmek	Ret, geçerli varyantları gizler; eşik ise modelin kendi bildirdiği bir sayıya güvenir
Tanımsız tipte tek adayın elenmesi ve bunun kaydedilmesi	Önceden olduğu gibi tüm yanıtı başarısız saymak	Geçerli adaylar korunur ve sapma sayılabilir
Yalnızca yeni kayıt eklenen kararlar ve hesaplanan kabul kayıtları	Grafın yeniden kurulduğu bir olay günlüğü	Denetim izi için yeterlidir; yeniden kurma ve geri alma gerçekleştirilmemiştir
Zincir segmenti başına tek çağrıyla, modelden önce kullanılabilen kural tabanlı bölütleyici	Zincirleri modelin bulduğu, hadis başına tek çağrı	Birebir alt dizgiler ve konumlar, birbirinden yalıtılmış başarısızlıklar ve modelsiz bir taban çizgisi sağlar. Değerlendirmede çıkarım kalitesine katkı bulunmamıştır (Bölüm 7.5); bu yüzden iki strateji de seçilebilir durumdadır
Koşu başına tek graf işlemi	Önceden olduğu gibi düğüm başına tek işlem	Başarısız commit, kısmi graf bırakmaz

Üç sınırı bilerek kabul ediyoruz. Ravi düğümü, harekeler dâhil tam yazılı adıyla tanımlanır; bu yüzden graf, aynı adı paylaşan kişileri birleştirir ve iki biçimde yazılan tek kişiyi ayırır. Rivayet metninin sınırından sonra başlayan zincirler bölütlenmez; çünkü oradaki aktarım edatları çoğunlukla anlatıya aittir. Kaynağa dayandırma kontrolleri yalnızca çıkarım çağrısından gelen adaylara uygulanır.

6 Gerçekleştirim

Bu bölümde `main` dalını esas alarak `giraffegraph-core` deposunun kodunu açıklıyoruz. Bu daldaki kodun büyük kısmı, yazarların yönlendirmesiyle yapay zekâ kodlama asistanları kullanılarak yazılmıştır ve Bölüm 6.7'de açıklanan otomatik test paketiyle doğrulanmaktadır.

6.1 Teknoloji yığını

Bileşen	Teknoloji	Bildirilen sürüm
Arka uç	Python, FastAPI, Uvicorn, Pydantic	Python `>=3.12` (Docker imajında ve CI'da 3,12), `fastapi>=0.115`, `uvicorn>=0.34`, `pydantic>=2.10`
Üst veri deposu	PostgreSQL, SQLAlchemy, psycopg, Alembic	imaj `postgres:16-alpine`, `sqlalchemy>=2.0`, `psycopg>=3.2`, `alembic>=1.15`
Graf deposu	Neo4j ve Python sürücüsü	imaj `neo4j:5`, `neo4j>=6.0,<7.0`
Model erişimi	`anthropic` SDK; standart kütüphane üzerinden OpenAI uyumlu HTTP API	`anthropic>=1.13,<2.0`
Panel	Next.js, React, TypeScript	`next ^15`, `react ^19`, `typescript ^5`; CI'da Node 22
Araçlar	uv, pytest, ruff, katı kipte mypy	`pytest>=8.3`, `ruff>=0.11`, `mypy>=1.15`

Sürümler, `backend/pyproject.toml`, `frontend/package.json` ve `docker-compose.yml` dosyalarında bildirilen kısıtlardır. Docker Compose beş servisi başlatır: `postgres`, `neo4j`, `backend`, `worker` ve `dashboard`.

6.2 Depo düzeni

```
backend/
  src/app/domain/          entities, segmentation, hadith helpers, identity keys
  src/app/application/     use cases and ports
  src/app/infrastructure/  LLM providers, template catalog, persistence
  src/app/interfaces/      API routers and worker CLI
  src/app/bootstrap/       application wiring
  migrations/versions/     Alembic migrations 0001 to 0006
  scripts/                 baseline and experiment runners
  tests/unit, tests/integration
frontend/src/modules/projects/   dashboard components, API client, types
evaluation/                scorer, analysis, annotation kit, their tests
```

6.3 API arayüzü

`interfaces/api/routers/` içindeki 13 yönlendirici, 33 `GET` ve 20 `POST` olmak üzere 53 uç nokta tanımlar.

Grup	Sayı	Örnek uç noktalar
Projeler, şablonlar	4	`POST /api/projects`, `GET /api/templates`
Şema	7	`POST /api/schema-versions/{schema_version_id}/node-types`
Boru hattı tanımları	3	`GET /api/schema-versions/{schema_version_id}/pipeline-definitions`

Grup	Sayı	Örnek uç noktalar
Veri alımı ve koşular	8	`POST /api/projects/{project_id}/source-units/bulk`, `POST /api/projects/{project_id}/source-units/process`, `POST /api/source-units/{source_unit_id}/runs`
Segmentler, adaylar, doğrulama	10	`POST /api/source-units/{source_unit_id}/segments/generate`, `POST /api/runs/{run_id}/validate`, `GET /api/runs/{run_id}/review-items`
Çıkarım	2	`POST /api/runs/{run_id}/extract-candidates`
İnceleme	4	`GET /api/projects/{project_id}/review-queue`, `POST /api/review-items/{review_item_id}/decision`, `GET /api/runs/{run_id}/acceptance-records`
Commit	2	`POST /api/runs/{run_id}/commit`, `GET /api/runs/{run_id}/graph-delta`
İşler	4	`POST /api/runs/{run_id}/processing-jobs`, `POST /api/processing-jobs/process-next`
Graf	5	`GET /api/projects/{project_id}/graph/nodes`
Zincirler	3	`GET /api/projects/{project_id}/chains/hadith`, `GET /api/projects/{project_id}/chains/narrator`
Sağlık	1	`GET /health`

Uygulama hataları, `error_handlers.py` içinde 404, 409, 422 ve 502 durum kodlarına eşlenir.

6.4 Model sağlayıcısı soyutlaması

`LLMProvider` portunun iki metodu vardır. `generate_structured` yanıtı sözlük olarak döndürür; `generate_structured_with_usage` ise yanıtla birlikte bir `LLMUsage` kaydı döndürür. `StaticLLMProviderRegistry`, sağlayıcıları adlarına göre tutar. `noop` (aday döndürmez) ve `rule_based_isnad` (Bölüm 5.8) her zaman kaydedilir; `openai` ve `anthropic` ise yalnız API anahtarları ayarlandığında kaydedilir.

- **OpenAI uyumlu** (`openai_provider.py`). `{base_url}/chat/completions` adresine, `temperature` değeri 0 ve `response_format` değeri `json_object` olacak biçimde istek gönderir; belirteç sınırında kesilen yanıtı hata sayar. Belirteç sayılarını raporlamaz.
- **Anthropic** (`anthropic_provider.py`). Messages API'yi bir `output_config` ile çağırır. Bu yapılandırma çaba ayarını ve bir JSON şemasını (`EXTRACTION_WIRE_SCHEMA`) taşır. Şema yalnız kapalı nesnelere izin verdiği için özellikler, ad ve değer girdilerinden oluşan bir liste olarak aktarılır ve sonra yeniden bir eşlemeye dönüştürülür. Çaba ayarı `ANTHROPIC_EFFORT` değişkeninden gelir (varsayılan `low`). Ret, kesilmiş yanıt ve her API hata sınıfı bir `StructuredGenerationError` olur; bu hata bir `kind` içerir. Kullanım kaydı; girdi ve çıktı belirteçlerini, önbellekten okunan ve önbellek oluşturmak için kullanılan belirteçleri, durma gerekçesini ve modeli tutar.

İstem kataloğu, tip adlarını eşleştirme kuralı bakımından farklılaşan `candidate_graph_extraction_v1` ve `candidate_graph_extraction_v2` sürümlerini tutar; varsayılan sürüm `v2`'dir.

`CachedLLMProvider`, bir sağlayıcıyı sarmalar ve her çağrı için bir JSON dosyası saklar. Anahtar; sağlayıcı adı, model, çaba, istem sürümü, istem metni ve veri yükünün SHA-256 özetidir. Özet alınmadan önce veri yükündeki koşu, şema sürümü, kaynak birimi ve segment kimlikleri, konumsal yer tutucularla değiştirilir. Bu nedenle aynı içerikle yapılan sonraki koşu önbellekte bulunur ve saklanan yanıt yeni koşunun segment kimliklerine yeniden eşlenir. Başarısız çağrılar saklanmaz.

Çıkarım kullanım durumu, `generate_structured` metodunu çağırır ve belirteç sayılarını saklamaz. Önbelleği ve kullanım kayıtlarını kullanan `scripts/run_isnad_experiment.py`, her çağrının belirteçlerini, gecikmesini ve maliyetini yazar; harcama sınırına ulaşıldığında çağrı yapmayı durdurur.

6.5 Kalıcı saklama

Postgres. ORM, 18 tablo tanımlar. `0001` göçü başlangıçtaki 15 tabloyu oluşturur; `0002` düğüm tipinin güven skoru eşiğini, `0003` `processing_jobs` tablosunu, `0004` `pipeline_definitions` tablosunu ekler. `0005`, `merge_keys` sütununu `relation_types` tablosuna ekler; `0006` ise iki indeksli `review_decisions` tablosunu ekler. `AUTO_APPLY_POSTGRES_MIGRATIONS` doğru olduğunda arka uç, başlangıçta göçleri uygular. Bir iş, `FOR UPDATE SKIP LOCKED` ve koşu başına bir danışma kilidi ile alınır; çalışan işi olan koşuya ikinci bir iş verilmez.

Neo4j. `commit_bundle`, `_write_bundle` fonksiyonunu `session.execute_write` metoduna geçirir; böylece bir koşunun bütün ifadeleri aynı veritabanı işlemini paylaşır. Önce kaynak birimini ve segmentlerini, sonra her düğümü (`project_id`, `node_type`, `merge_key`) üzerinden `CanonicalNode` ve tip adı etiketleriyle, ardından her ilişkiyi kimlik anahtarıyla birleştirir. Köken bilgisi; kaynak birimine `DERIVED_FROM`, segmentlere `SUPPORTED_BY`, commit edilen her ilişki adayı için bir `RelationEvidence` düğümü ve bir `GraphDelta` düğümü olarak yazılır. Tip adları bir ifadeye girmeden önce tanımlayıcı örüntüsüne göre kontrol edilir. Bir sürecin ilk commit işleminde `CREATE INDEX ... IF NOT EXISTS` ile beş indeks oluşturulur: düğüm anahtarı için bir indeks, `SourceUnit`, `Segment`, `RelationEvidence` ve `GraphDelta` tiplerinin id özelliği için de birer indeks. Benzersizlik kısıtı tanımlanmamıştır. Alıntılar ve konum ofsetleri Postgres'teki aday satırlarında kalır; graf, segmentlerin bütününe bağlanır.

6.6 Panel

Panel, geliştirici olmayan bir okuyucu için yazılmış Türkçe bir Next.js uygulamasıdır. Renkleri ve yazı biçimi, projenin kamuya açık sayfasını izler. Ana sayfa ürünü iki cümleyle açıklar ve projeleri, durum cümlesi içeren kartlar olarak listeler; örneğin "30 hadis · hepsi grafta · bekleyen yok" (30 hadis, hepsi grafta, bekleyen yok). Proje oluştururken yalnız ad sorulur; isnad şablonu ve şeması otomatik uygulanır.

Bir proje, sayılarla birlikte adım göstergesinde sunulan dört adımdan oluşur. Her adımın bir ekranı ve bir ana eylemi vardır.

1. **Metin yükle (metin yükleme).** Kullanıcı metin yapıştırır, bir JSON dosyası seçer ya da paketle gelen on örnek hadisi tek tıklamayla yükler. Sonuç, yinelenen kayıtlar da dahil olmak üzere bir cümleyle bildirilir.
2. **Zincirleri çıkar (zincir çıkarımı).** Kullanıcı bir yöntem seçer: kurallar ya da sunucuda yapılandırılmışsa bir dil modeli. Tek bir düğme, bekleyen bütün hadisleri işler ve kaç hadisin grafa yazıldığını, kaçının karar beklediğini özetler.
3. **İncele (inceleme).** Yalnız bir kişinin kararını gerektiren öneriler, teker teker görünür: alıntısı vurgulanmış hadis metni, önerilen değer, neden beklediğini açıklayan bir cümle ve üç eylem (kabul, ret, düzeltme). Her inceleme gerekçe kodunun kendi cümlesi vardır; değer kontrolü için "önerilen ad, sistemin gösterdiği alıntıda geçmiyor" yazar. İnceleyicinin adı bir kez sorulur ve tarayıcıda saklanır; bir öğenin açılması ile karar verilmesi arasında geçen süre kararla birlikte gönderilir.
4. **Zincirleri oku (zincir okuma).** Commit edilen hadisler bir arama alanıyla listelenir. Seçilen hadisin Arapça metni sağdan sola gösterilir ve her ravi geçişi, kaynaktaki konum ofsetlerinde vurgulanır. Her zincir yukarıdan aşağıya okunur: önce ravi, ardından aktarım ifadesi, alıntısı ve nasıl kabul edildiğiyle birlikte bağlantı gelir. Kabul biçimi açıkça "otomatik kabul edildi (kontrolleri geçti)" ya da ad ve tarihle birlikte "bir kişi tarafından kabul edildi" olarak yazılır. Bir ravi seçildiğinde, adın geçtiği hadisler ile kendisine aktaran ve kendisinin aktardığı kişilerin adları gösterilir; ravinin yazılı adla tanımlandığını belirten bir not bulunur.

Şema düzenleyicisi, adım adım boru hattı çalıştırıcısı, iş listesi ve ham graf gezgini, ikincil bir giriş olan "Gelişmiş" (gelişmiş seçenekler) altında yer alır. Beş terimlik sözlüğe her sayfadan ulaşılır. Her ekranın görüntüleri `frontend/docs/screens/` içindedir.

6.7 Testler ve sürekli entegrasyon

`uv run pytest --collect-only -q`, 285 test toplar: 14 dosyada 127 birim testi ve 21 dosyada 158 entegrasyon testi. `uv run pytest -q` koşumuzda 273 test geçti, 12 test atlandı. Entegrasyon testleri, önceden tanımlanmış yanıtlar veren sağlayıcılarla bellek içi arka uçta, FastAPI'nin `TestClient` aracı üzerinden API'yi çalıştırır; dolayısıyla hiçbir test bir ağ servisini çağırmaz.

Atlanan 12 test, `real_db` işaretini taşır. `test_realdb_migrations.py` (4 test), göç zincirini boş bir veritabanında çalıştırır, sonucu ORM modelleriyle karşılaştırır ve sürüm düşürmeyi kontrol eder. `test_realdb_isnad_flow.py` (7 test), isnad akışını Postgres ve Neo4j üzerinde çalıştırır: iki hadiste aynı ravi çifti iki kenar üretir, grafın tekrar yazılması hiçbir şey eklemez, kesintiye uğrayan yazma işlemi kısmi graf bırakmaz ve arama indeksleri bulunur. `test_realdb_review_decisions.py` (1 test), kararların saklandığını ve eklenerek ilerlediğini kontrol eder. Bu testler yalnız `GG_TEST_POSTGRES_DSN`, `GG_TEST_NEO4J_URI` ve `GG_TEST_NEO4J_PASSWORD` ayarlandığında çalışır; geçici Postgres veritabanları oluşturup silerler. 12 Ekim 2026'da yerel konteynerlerde PostgreSQL 16 ve Neo4j 5,26 üzerinde çalıştırıldılar ve 12'si de geçti; aynı gün, o sırada mevcut olan 214 test de bu veritabanlarında bir kez çalıştırıldı ve geçti. Bu doğrulamada, bellek içi arka ucun gizlediği dört kusur bulundu. Bunlar arasında düğüm başına bir veritabanı işlemi kullanan ve kısmi graf bırakabilen bir graf yazma işlemi ile Neo4j indekslerinin hiç bulunmaması vardı. Dördü de düzeltildi.

Değerlendirme kitinin `evaluation/tests/` içinde kendi testleri vardır: sekiz dosyada 81 test. 80'i geçer; biri, puanlayıcıyı pilot betiğiyle karşılaştırdığı için `numpy` ve `scipy` kurulu olmadıkça atlanır. Panelin otomatik testi yoktur.

CI iş akışı, motorun klasörünü değiştiren her push ve pull request için iki iş çalıştırır. Arka uç işi `ruff check src tests`, `mypy src` ve `pytest -q` komutlarını; ön uç işi `npm ci` ve `npm run build` komutlarını çalıştırır. CI'da veritabanı değişkenleri ayarlanmadığından `real_db` testleri atlanır; değerlendirme kitinin testleri de çalıştırılmaz.

6.8 Çalıştırma

Deponun kökünden bütün yığını çalıştırmak için:

```
cp .env.example .env
docker compose up --build
```

Ardından panel `http://localhost:3000`, API ise `http://localhost:8000` adresindedir.

Arka uç testleri ve kontrolleri:

```
cd backend
uv sync --extra dev
uv run pytest -q
uv run ruff check src tests
uv run mypy src
```

`docs/guides/isnad-demo.md` kılavuzuna göre, veritabanı ya da API anahtarı olmadan isnad demosu:

```
cd backend
REPOSITORY_BACKEND=memory ALLOWED_ORIGINS=http://localhost:3000 \
  uv run uvicorn app.main:app --app-dir src --port 8000
```

```
cd frontend
NEXT_PUBLIC_API_BASE_URL=http://localhost:8000 npm run dev
```

Kılavuz daha sonra `isnad` şablonundan bir proje oluşturur, toplu yükleme uç noktasından hadisleri yükler, `rule_based_isnad` sağlayıcısıyla işler ve zincirleri okur.

6.9 Bilinen sınırlamalar

Üç yetenek, alınan karar gereği yoktur: commit edilen koşu geri alınamaz, kimlik doğrulama yoktur ve ravi kimliği çözümlemesi yoktur.

Kodun ayrıca şu sınırları vardır.

- Graf değişimi satırı, Neo4j veritabanı işleminden sonra Postgres'e yazılır. İkisi arasında bir hata olursa graf, üst verinin önüne geçer.
- Düğüm özellikleri `SET n += $properties` ile yazılır; bu nedenle sonraki bir koşu, farklı bir değerin üzerine inceleme öğesi oluşturmadan yazar.
- Şema önerisini kabul etmek inceleme öğesini kapatır, şemayı değiştirmez.
- İşleme uç noktası istek içinde çalışır; zincir uç noktaları projenin bütün kaynak birimlerini ve koşularını tarar.
- İlişki ayrıntısı uç noktasında kimlik anahtarı parametresi bulunmaz; bu yüzden paralel kenarlardan birini döndürür.
- Segmentlerin yeniden üretilmesi, saklanan segmentleri yeni kimliklerle değiştirir; dolayısıyla önceki koşuların kanıt başvuruları artık var olmayan segmentleri gösterir.
- Bir önerinin birden fazla gerekçesi varsa panel yalnız ilkini gösterir, proje sayılarını proje başına iki istekle tarayıcıda hesaplar ve otomatik testi yoktur.
- Henüz bir lisans seçilmemiştir.

7 Değerlendirme

Bu bölümde neyi, hangi veriler üzerinde, hangi referansa karşı ve hangi eşleştirme kuralıyla ölçtüğümüzü raporluyoruz. Ölçülen sayılar, Bölüm 6'da açıklanan betiklerin ürettiği `giraffegraph-core/evaluation/results/` altındaki dosyalardan; veri kümelerinin istatistikleri ise veri manifestosundan gelir [6]. Bütün koşuları 12 Ekim 2026'da motorun bellek içi arka ucunda gerçekleştirdik. PostgreSQL ve Neo4j arka ucu testlerle ayrıca doğrulandı (Bölüm 6.7); bu koşularda kullanılmadı.

7.1 Sorular

Değerlendirme beş soruyu yanıtlar.

1. Her çıkarım koşulu, bir hadisin ravi geçişlerini ne ölçüde elde ediyor?
2. Model ile graf arasına yerleştirilen doğrulama kontrolleri neyi yakalıyor ve bunların doğru çıktı kaybı açısından maliyeti nedir?
3. Modelin bildirdiği güven skoru, bir kişinin neyi incelemesi gerektiğine karar vermek için kullanılabilir bir sinyal midir?
4. Her koşul ne kadar inceleme işi ve parasal maliyet doğuruyor?
5. Yazılı bir ad, bir raviyi ne ölçüde tanımlıyor?

Yanıtlayamadığımız soruları Bölüm 7.11'de listeliyoruz.

7.2 Veriler ve referans etiketleri

İki hadis kümesi kullanıyoruz (Tablo 7.1). İkisinin de iki bağımsız etiketleyiciyle oluşturulmuş bir insan altın standardı yoktur; hiçbir kaynak da bu etiketleri böyle tanımlamaz [6].

Tablo 7.1 Değerlendirme kümeleri.

Küme	Hadis sayısı	Referans geçişleri	Metin ve etiketlerin kaynağı	İşlev
Pilot	50, beş kitap, Bukhari yok	327	Sanadset 650K [5]; etiketler düzenli ifadelerle üretilmiş	Geliştirme kümesi. Kural tabanlı bölütleyici bu küme üzerinde ayarlandı
Altı kitap	300, altı temel hadis kitabının her birinden 50	1.778	Ifta Sunnah veri ailesi [1, 6]; etiketler, Ifta platformunun üçüncü bir tarafça yeniden yapılandırılmış işaretlemeleridir	Test kümesi. Metni bir pilot hadisle eşleşen bütün kayıtlar çıkarıldıktan sonra sabit tohumla seçildi

Altı kitap kümesini, uzunluğu 100 ile 3.000 karakter arasında olan ve en az iki etiketli ravi içeren kayıtlardan örnekledik. Normalizasyondan sonra her referans geçişi kendi hadis metninde bulunur ve 300 hadisin tamamında geçişler metindeki sırayı izler. Bir hadiste 2 ile 11 arasında geçiş vardır (medyan 6); 300 hadisin 74'ü birden fazla zincir içerir.

Platform etiketlerinde bilinen tutarsızlıklar vardır. Derleyicinin açıklaması ya da destekleyici bir zincir içindeki raviler bazen etiketlenir, bazen etiketlenmez; dolayısıyla bunları çıkaran bir sistem bazı hadislerde yanlış sayılır. Etiketleri düzeltmedik.

Düzenli ifadelerle üretilen etiketler ne kadar iyi? 300 hadisin 253'ü için Sanadset'te eşleşen bir kayıt vardır (metin benzerliği en az 0,9; 118'i normalizasyondan sonra aynıdır). Sanadset etiketlerini Ifta etiketlerine karşı puanladığımızda gevşek eşleştirmede precision 0,939, recall 0,979 ve F1 0,959 (%95 aralığı 0,943 ile 0,972 arasında), katı eşleştirmede ise F1 0,931 elde ettik. İki etiket kümesi bu hadislerde birbirine yakındır. Bu, iki etiket kümesinin uyumudur, herhangi birinin doğruluğu değildir. Ayrıca bu 253 hadis, rastgele bir alt küme değil, metni her iki derlemde eşleşen hadislerdir.

7.3 Protokol

Analiz birimi hadistir. Her hadis için sistemin ravi geçişlerini, en büyük ağırlıklı iki parçalı eşleştirmeyle referans geçişlerine hizalıyoruz; eşleşen çiftleri doğru pozitif sayıyoruz. Koşulardan önce sabitlediğimiz iki eşleştirme kuralını raporluyoruz:

- **Katı:** Normalize dizgiler eşittir. Normalizasyon harekeleri ve noktalama işaretlerini kaldırır, harf varyantlarını birleştirir.
- **Gevşek:** Dizgiler eşittir, biri diğerini içerir ya da belirteç F1 değerleri en az 0,60'tır.

Precision, recall ve F1 için hadisler üzerinden mikro ortalama alıyoruz. Aralıklar, hadislerin 2.000 kez yeniden örneklenmesiyle elde edilen %95 yüzdelik bootstrap aralıklarıdır. İki koşulun farkını hesaplarken ikisi için de aynı yeniden örneklemeleri kullanıyoruz. Sistemin hiçbir şey döndürmediği hadis sıfır geçiş olarak sayılır ve hiçbir zaman dışarıda bırakılmaz.

Bu bölüm boyunca "yanlış" geçiş, gevşek kurala göre bir referans geçişiyle eşleşmeyen önerilen geçiş; "doğru" geçiş ise eşleşen geçiştir. Her iki ifade de etiketlerin yüzey biçimiyle uyuma ilişkindir. Bölüm 7.6, eşleşmeyen geçişlerin nelerden oluştuğunu gösterir.

Katı eşleştirme, geçişin nerede bittiğine duyarlıdır. Karşılaştırmalarda gevşek kuralı kullanıyoruz; iki ölçü arasındaki fark da bilgi verdiği için yanında katı F1'i de sunuyoruz [31].

7.4 Koşullar

Bütün koşullar aynı motordan geçer: veri alımı, bölütleme, sağlayıcı çağrısı, doğrulama ve commit. Hiçbir koşuda inceleme kararı verilmediği için sonuçlar yalnız boru hattının otomatik kısmını gösterir.

Tablo 7.2 Çıkarım koşulları.

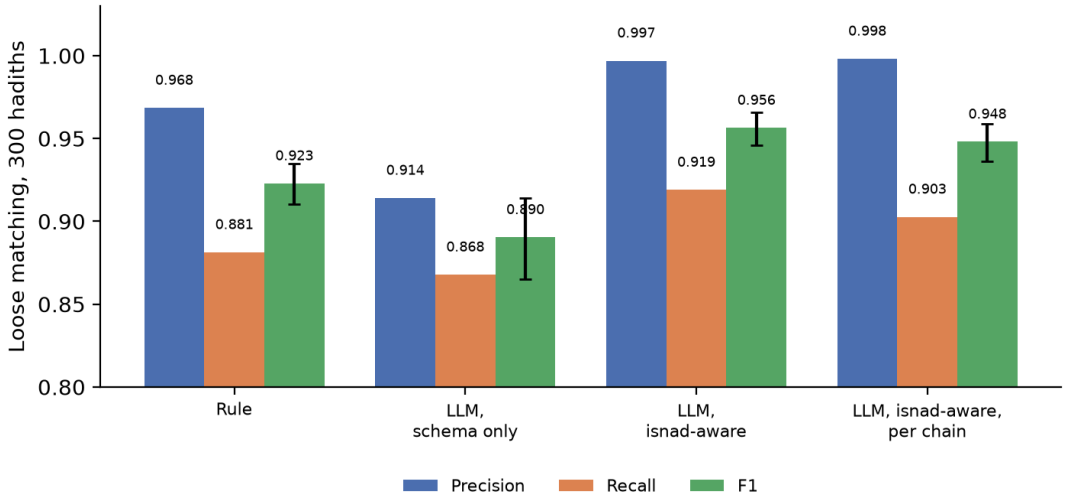
Koşul	Bölütleme	Sağlayıcı	Şema ipuçları
Kural	Kural tabanlı bölütleyicinin bulduğu zincir metin aralığı, ح işaretinde bölünür	`rule_based_isnad`, model yok	kullanılmaz
Model, yalnız şema	Hadisin tamamı, tek çağrı	Claude Haiku 5.5	Yalnız tip adları ve JSON sözleşmesi; tip ve özellik açıklamaları çıkarılmış
Model, isnad bilgili	Hadisin tamamı, tek çağrı	Claude Haiku 5.5	Açıklamalar ve şablon ipuçları: adları aynen kopyala, aktarım fiillerini dışarıda bırak, Peygamber ravi değildir
Model, isnad bilgili, zincir metin aralığı	Bölütleyiciden gelen zincir metin aralığı, segment başına bir çağrı	Claude Haiku 5.5	Aynı açıklamalar ve ipuçları

Model, `low` çaba ayarıyla ve yapılandırılmış çıktıyla çalışır. Yanıtlar içerikle adreslenen bir önbellekte saklandığından her koşu yeni çağrı yapılmadan yeniden üretilebilir.

7.5 Çıkarım sonuçları

Tablo 7.3 Altı kitap test kümesinde ravi geçişi çıkarımı (300 hadis, 1.778 referans geçişi, Ifta etiketleri). P, R ve F1 gevşek eşleştirmeyi kullanır; aralık, gevşek F1 içindir.

Koşul	P	R	F1 [%95 aralığı]	Katı F1	Eksiksiz zincirler	TP / FP / FN
Kural	0,968	0,881	0,923 [0,910, 0,935]	0,633	0,490	1.567 / 51 / 211
Model, yalnız şema	0,914	0,868	0,890 [0,865, 0,914]	0,826	0,567	1.543 / 145 / 235
Model, isnad bilgili	0,997	0,919	0,956 [0,946, 0,966]	0,904	0,690	1.634 / 5 / 144
Model, isnad bilgili, zincir metin aralığı	0,998	0,903	0,948 [0,936, 0,959]	0,897	0,650	1.605 / 3 / 173



Şekil 7.1 Altı kitap test kümesinde koşula göre gevşek precision, recall ve F1. F1 üzerindeki hata çubukları %95 bootstrap aralıklarıdır.

"Eksiksiz zincirler", gevşek eşleştirmede her geçişin sırayla eşleştiği ve fazladan hiçbir geçişin bulunmadığı hadislerin payıdır.

Gevşek eşleştirmede tek başına şema, modeli kurallardan daha iyi yapmaz. Yalnız şema verilen model, kural tabanlı ayrıştırıcının F1'de 0,032 (aralık 0,006 ile 0,060 arasında), precision'da 0,054 (0,026 ile 0,083 arasında) altındadır. Recall farklı değildir (0,013, aralık −0,018 ile 0,044 arasında). Katı eşleştirmede sıralama tersine döner: katı F1, yalnız şema verilen model için 0,826, ayrıştırıcı için 0,633'tür. Model ayrıca daha fazla zinciri bütünüyle doğru çıkarır (0,490'a karşı 0,567). Ayrıştırıcı doğru adları bulur ama metin aralıkları çoğu zaman adın sonunu aşar; model temiz adlar döndürür, bazı adları da ekler ya da değiştirir.

Kazanımın büyük kısmı açıklamalardan ve ipuçlarından gelir. Aynı modele tip ve özellik açıklamaları ile isnad ipuçlarını vermek precision'ı 0,083 (0,059 ile 0,108 arasında), recall'u 0,051 (0,024 ile 0,078 arasında), F1'i 0,066 (0,041 ile 0,091 arasında) artırır. Eşleşmeyen öneriler 145'ten 5'e düşer. İpuçlarıyla model, kural tabanlı ayrıştırıcının gevşek F1'de 0,034 (0,021 ile 0,047 arasında) üzerindedir; katı eşleştirmede ise çok daha üstündedir. Şemasız bir koşul çalıştırmadık. Bu nedenle ölçtüğümüz, şema verildiğinde talimatların etkisidir; talimatlarla şemanın karşılaştırması değildir.

Modeli zincir metin aralığıyla sınırlamak yardımcı olmadı. Dördüncü koşulda model yalnız bölütleyicinin bulduğu zincir metin aralığını görür. F1, hadisin tamamı için 0,956 iken bu koşulda 0,948'dir;

0,008'lik farkın aralığı −0,002 ile 0,019 arasındadır. Bölütleyici yalnız zincir değişimi işaretinde yeni segment başlattığından 300 hadisin yalnız 10'u iki çağrıya bölündü. Dolayısıyla koşul, zincir başına bir çağrıdan çok girdinin sınırlandırılmasını sınar. Birden fazla zincir içeren uzun hadislerde zincir başına çağrıların yardımcı olup olmadığı sorusu açık kalır. İki koşulda da hiçbir çağrı kesilmedi. 30 Eylül'deki tasarım analizi, farklı bir modelde kesilmiş yanıtlar gördükten sonra zincir başına çağrı önermişti; Claude Haiku 5.5 ve bu uzunluktaki hadislerle kesilme gözlemlemiyoruz. Zincir metin aralığı koşulunda, motorun tanımsız bir düğüme başvuran ilişkiyi reddetmesi yüzünden 310 çağrıdan biri başarısız oldu.

Tablo 7.4 Hadisin yapısına göre gevşek recall (altı kitap kümesi).

Koşul	1 zincir (226)	2+ zincir (74)	2–4 geçiş (22)	5–6 geçiş (203)	7–11 geçiş (75)
Kural	0,924	0,778	0,843	0,923	0,806
Model, yalnız şema	0,886	0,824	0,928	0,876	0,844
Model, isnad bilgili	0,940	0,868	0,976	0,948	0,854
Model, isnad bilgili, zincir metin aralığı	0,915	0,874	0,964	0,930	0,841

Bütün koşullarda paralel zincirli ve uzun hadislerde recall düşer. Paralel zincirlerde en büyük kayıp kural tabanlı ayrıştırıcıdadır (0,924'ten 0,778'e). Kitaplara göre gevşek F1, isnad bilgili model için 0,936 (Bukhari) ile 0,980 (Nasa'i), kural tabanlı ayrıştırıcı için 0,862 (Abu Dawud) ile 0,963 (Nasa'i) arasında değişir.

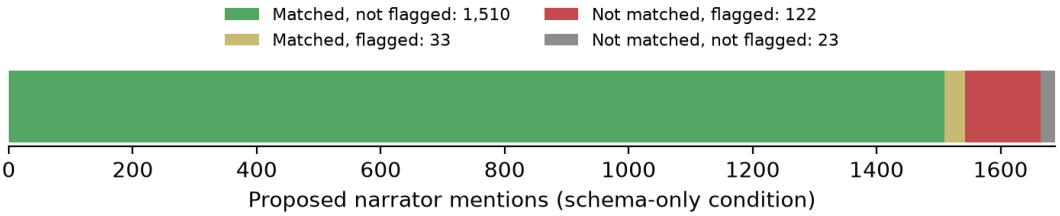
Sanadset etiketlerine karşı puanlanan pilot kümede koşulların sıralaması aynıdır: kural 0,892, yalnız şema 0,828, isnad bilgili 0,945, zincir metin aralığı 0,919 (gevşek F1). Kural tabanlı ayrıştırıcı bu 50 hadis üzerinde ayarlandığı için pilot sayıları onun lehine iyimserdir.

7.6 Kontrollerin yakaladıkları

Kontroller, modelin metinden saptığı koşulda, yani yalnız şema verilen koşulda etkili olur. Tablo 7.5, önerilen geçişin referansla eşleşip eşleşmediğini ve bir kontrolün onu işaretleyip işaretlemediğini çaprazlar.

Tablo 7.5 Yalnız şema verilen koşulda değer-alıntıda kontrolü (altı kitap kümesi, önerilen 1.688 geçişin 145'i referansla eşleşmiyor).

	Eşleşmeyen	Eşleşen
Kontrolün işaretledikleri	122	33
İşaretlenmeyenler	23	1.510



Şekil 7.2 Yalnız şema verilen koşulda önerilen geçişler: referansla eşleşme ve değer-alıntıda kontrolünün işaretleyip işaretlememesi.

Kontrol, 70 hadiste 155 geçişi (önerilerin %9,2'sini) işaretler. İşaretlenen geçişlerin %79'u eşleşmeyendir ve bunlar bütün eşleşmeyen geçişlerin %84'ünü oluşturur. Düğüm düzeyindeki diğer kontrollerin hiçbiri tetiklenmedi: modelin verdiği her alıntı, kontrollerin kullandığı karşılaştırmaya göre kaynak segmentinde bulundu.

Kontrolün saptadığı, modelin alıntıladığı metinde bulunmayan bir ad yazmasıdır. Tablo 7.6, işaretlenen 155 adı sabit kurallarla sınıflandırır (`evaluation/classify_value_flags.py`).

Tablo 7.6 Yalnız şema verilen koşulda işaretlenen adların değişiklik türleri.

Değişiklik türü	Örnek (alıntı, ad)	İşaretlenen	Eşleşmeyen	Eşleşen
Latin harflerine çevriyazı	حَدَّثَنَا الْأَعْمَشُ, "al-A'mash"	79	79	0
Adın ad hâli değiştirilmiş	عَنْ أَبِي بِشْرٍ (tamlayan hâli), أَبُو بِشْرٍ (yalın hâl)	34	21	13
Akrabalık ifadesi adla değiştirilmiş	عَنْ أَبِيهِ ("babasından"), عُرْوَةَ	25	18	7
Diğer, ad metnin başka yerinde bulunuyor	ad alıntıdan daha uzun	7	3	4
Diğer, ad metinde bulunmuyor	حَدَّثَنَا يَحْيَى ، هُوَ ابْنُ سَعِيدٍ, يحيى بن سعيد	10	1	9
Toplam		155	122	33

Üç gözlem, öne çıkan sayıların nasıl okunacağını sınırlar.

Birincisi, "yanlış" geçişlerin çoğu uydurulmuş raviler değildir. Yetmiş dokuzunda doğru ravi Latin harfleriyle yazılmıştır; hepsi modelin yazı değiştirdiği 14 hadisten gelir. Yirmi birinde doğru ravi farklı bir ad hâlindedir. Hâl değişikliğinin eşleşme sayılıp sayılmaması adın uzunluğuna bağlıdır: iki sözcüklü bir adda belirteç örtüşmesi gevşek kuralın eşiğinin altına düşer, daha uzun bir adda düşmez. Kontrol de puanlayıcı da yüzey biçimlerini karşılaştırdığı için uyumlarının bir kısmı yapılarından kaynaklanır.

İkincisi, akrabalık örnekleri uydurma değil, çıkarımdır. Örnekte metin "Hisham ibn Urwa, babasından" der; model ise "Urwa" yazmıştır. Ad metinde vardır, alıntıda yoktur. 25 akrabalık adının 15'i hadis metninde geçer. Metin dışından bilgi gerektiren, işaretlenmiş bir ad bulduk: alıntı قَالَ عُثْمَانُ, ad ise عثمان بن عفان olup ikinci kısım hadisin hiçbir yerinde geçmez.

Üçüncüsü, Bölüm 2.3'teki sınıflar şöyle ayrılır. Eşleşmeyen 145 geçişin 102'sinde, normalize hadis metninde bulunmayan bir ad vardır; kontrol bu 102 geçişin hepsini işaretler. Kırk üçünde ad metinde bulunur; kontrol bunların 20'sini, yani adı alıntının dışında kalanları işaretler. Eşleşmeyen on geçiş, Peygamber'i ravi olarak adlandırır; bu bir kategori hatasıdır. Kontrol bunların ikisini, ikisi de Latin harfleriyle yazılmış olanları işaretler; ad metinden doğru kopyalandığı için sekizini kaçırır. Gerçekleştirilmiş hiçbir kontrol bu sınıfı kapsamaz.

İşaretlenen aday, bir kişi karar verene kadar kabul edilmediğinden bu koşulda incelemesiz kabul edilen adayların precision'ı 0,914 yerine 0,985, recall'u ise 0,868 yerine 0,849'dur (Tablo 7.7). Grafın kendisine daha az çıktı ulaşır: açık inceleme öğesi olan koşu commit edilmez ve 300 hadisin 110'unda böyle bir öğe vardır. Bu hadisleri boş saydığımızda, bir inceleyici karar verene kadar commit edilen çıktının precision'ı 0,983, recall'u 0,582'dir.

Tablo 7.7 Yalnız şema verilen koşulun üç noktadaki çıktısı (altı kitap kümesi, gevşek eşleştirme).

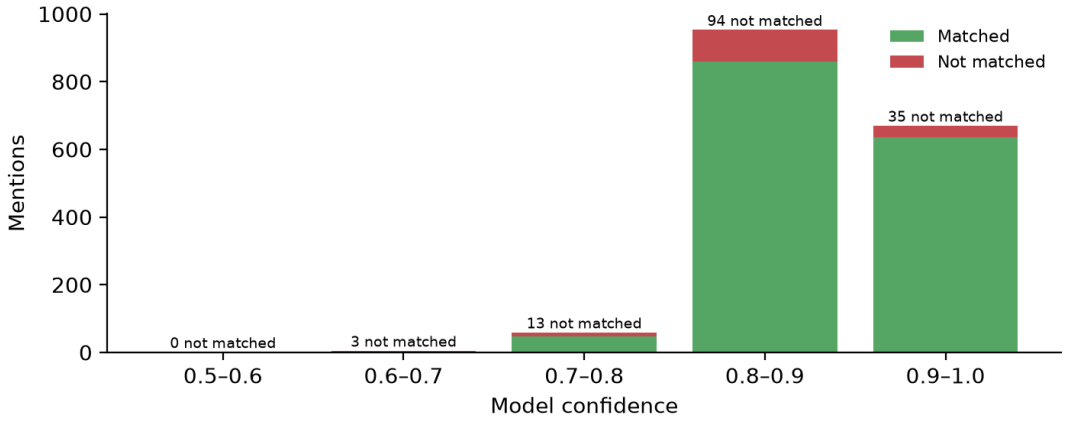
Çıktı	P	R	F1	TP / FP / FN
Modelin önerdiği her şey	0,914	0,868	0,890	1.543 / 145 / 235
İncelemesiz kabul edilen adaylar	0,985	0,849	0,912	1.510 / 23 / 268

Çıktı	P	R	F1	TP / FP / FN
İncelemesiz grafa commit edilenler (190 hadis)	0,983	0,582	0,731	1.034 / 18 / 744

İsnad bilgili koşulda 1.639 önerinin 5'i eşleşmeyendir; kontrol bunların 3'ünü işaretler ve eşleşen hiçbir geçişi işaretlemez. İstem zaten modele adları yazıldığı gibi kopyalamasını söylediğinde kontrollerin bulacağı pek az şey kalır. Pilot kümede, Sanadset etiketlerine karşı yalnız şema verilen koşulun örüntüsü tekrarlanır: 35 geçiş işaretlenir, 35'inin tamamı eşleşmeyendir, eşleşmeyen 3 geçiş ise işaretlenmez.

7.7 Modelin güven skoru

Yalnız şema verilen koşulda, eşleşen geçişi öngörmek için modelin güven skorunun ROC eğrisi altında kalan alanı 0,634'tür. Model yedi farklı güven skoru kullanmıştır; 1.688 geçişin 1.623'ü 0,8 ile 1,0 arasında bir değer taşır.



Şekil 7.3 Yalnız şema verilen koşulda eşleşen ve eşleşmeyen geçişlerin model güven skorları.

Bir güven skoru eşiği ile bir kontrol, ancak incelemeye gönderilen geçişlerin payı aynı olduğunda karşılaştırılabilir. Tablo 7.8, anlamlı bir pay seçen üç eşiği listeler.

Tablo 7.8 Güven skoru eşiğinin ve değer-alıntıda kontrolünün incelemeye gönderdiği eşleşmeyen geçişler (yalnız şema verilen koşul, toplam 145 eşleşmeyen geçiş).

Sinyal	İncelemeye gönderilen geçişler	Bunların içinde eşleşmeyenler
Güven skoru 0,80'in altında	65 (%3,9)	16
Güven skoru 0,85'in altında	275 (%16,3)	50
Güven skoru 0,90'ın altında	1.018 (%60,3)	110
Değer-alıntıda kontrolü	155 (%9,2)	122

Güven skoru eşiği, geçişlerin %16'sında eşleşmeyen 50 geçiş bulur; kontrol ise %9'unda 122 geçiş bulur. Eşiğin 110 geçişi bulması için bütün geçişlerin %60'ını inceleyiciye göndermesi gerekir. Kontrol, model çağrısı gerektirmez.

7.8 İnceleme yükü, commit işlemleri ve maliyet

Tablo 7.9 İnceleyiciye kalan iş ve koşul başına maliyet (altı kitap kümesi).

Koşul	İncelemeye gönderilen adaylar	İncelenecek öğesi olmayan hadisler	İncelemesiz commit edilen koşular	1.000 hadis başına maliyet	Çağrı başına medyan gecikme
Kural	0	%100	300'ün 300'ü	0	uygulanamaz
Model, yalnız şema	538	%63	300'ün 190'ı	USD 1,87	11,1 sn
Model, isnad bilgili	16	%98	300'ün 294'ü	USD 2,29	13,0 sn
Model, isnad bilgili, zincir metin aralığı	6	%99	300'ün 295'i	USD 2,21	12,8 sn

Yalnız şema verilen koşuldaki 538 inceleme adayı, değer kontrolünün işaretlediği 155 düğüm ve 383 ilişkiden oluşur. İlişki bulguları arasında kimlik özelliği değeri bulunmayan 192, incelemedeki bir düğüme bağlı 208 ve güven skoru eşiğinin altında kalan 9 ilişki vardır; bir ilişki birden fazla bulgu taşıyabilir. Açık inceleme öğesi olan koşu commit edilemediğinden 300 hadisin 110'u bir kişiyi bekler.

Maliyetleri, deney betiğindeki fiyat sabitleri ve API'nin döndürdüğü belirteç sayılarıyla hesapladık. Sabitler, sağlayıcının 6 Ekim 2026 tarihli Claude Haiku 5.5 fiyat listesinden alınmıştır (bir milyon girdi belirteci için USD 0,10, bir milyon çıktı belirteci için USD 0,50). Hadis başına yaklaşık 3.200 ile 3.500 girdi, 3.100 ile 3.900 çıktı belirteci kullanılmıştır. Gecikme, yanıt önbelleğinden sunulmayan çağrıların medyanıdır (300, 300 ve 108 çağrı). Pilot koşular ve Bölüm 7.9'daki alt küme koşuları dahil bütün değerlendirmenin maliyeti, önbellekten yeniden oynatmalar sayılmadan sekiz model koşusu üzerinden toplandığında USD 3,49'dur.

Her koşuda commit edilen her olgu için iki özelliği kontrol ettik: alıntının, kontrollerin kullandığı karşılaştırmaya göre kaynak segmentinde bulunması ve bir kabul kaydının olması. İkisi de commit edilen bütün olgular için sağlanır. Bayt düzeyinde tam eşitlik karşılaştırmasında model koşuları ilkini sağlamaz: model Arapça metni kaynaktan farklı bir Unicode bileşimiyle döndürür; birkaç alıntı da kaynaktan, karşılaştırmanın yok saydığı harekeler bakımından farklıdır.

İnsan inceleme süresine ilişkin ölçümümüz yoktur. Karar kaydı bu süreyi saklar, ancak bu koşuları hiçbir kişi incelemedi.

7.9 İkinci bir model: yapılandırma hatası

Sonuçların modele bağlı olup olmadığını görmek için Claude Sonnet 5.5'i aynı ayarlarla, tabakalı örneklenmiş 36 hadislik bir alt kümede (kitap başına altı hadis) çalıştırdık. Koşu, kullanılabilir bir karşılaştırma üretmedi; nedenini raporluyoruz.

Sonnet 5.5, `low` çaba ayarıyla isnad bilgili koşulda 36 hadisin 28'i için tek ravi döndürdü; bunların çoğunda ravinin adını bir özelliğin anahtarını tutan alana yazdı. Gevşek recall, aynı 36 hadiste Haiku 5.5 için 0,916 ve 0,828 iken Sonnet için 0,237 (isnad bilgili) ve 0,530 (yalnız şema) oldu.

Yapılandırmamızda iki şeyden şüpheleniyoruz; ikisini de ek bir koşuyla sınamadık. Sağlayıcı, özellikleri `name` ve `value` alanlarını içeren bir girdi listesi olarak gönderir; ravi adını tutan özelliğin adı da `name` olduğundan bu belirsizlik yaratır. İkincisi, çaba ayarı modeller arasında aynı biçimde kalibre edilmemiştir. Haiku 5.5, altı kitap kümesindeki 910 çağrısının biri dışında hepsinde biçime uydu.

Motor, bozuk biçimli çıktıyı tasarlandığı gibi ele aldı. Tanımsız özellik anahtarları şema önerisi olarak kaydedildi (bir koşulda 29 farklı anahtar), kimlik özelliği olmayan düğümler reddedildi, 36 koşunun 30'unun commit edilmesi engellendi ve önerilen çıktının precision'ı 0,966 ya da üzerinde kaldı. Bozuk biçimli hiçbir çıktı grafa ulaşmadı. Bu, ilk kodun tip düzeyindeki kontrolünün göremediği, özellik düzeyindeki şema sapmasıdır.

Bu koşudan iki model hakkında sonuç çıkarmıyoruz. Başka bir model değerlendirilmeden önce aktarım biçimi değiştirilmelidir.

7.10 Ravi kimliği

Motor, ravi düğümünü yazılı adla tanımlar. Ifta etiketleri her geçişe bir kişi kimliği bağlar; böylece hiçbir sistem çalıştırmadan bu seçimin maliyetini ölçebiliriz. Etiketler her zinciri ayrı listeler; dolayısıyla paralel zincirlerin paylaştığı ravi, zincir başına bir kez sayılır. 300 test hadisi boyunca bu, 740 kişinin doldurduğu 2.150 zincir konumu verir (Tablo 7.1'deki 1.778 referans geçişi, her yazılı geçişi bir kez sayar).

Tablo 7.10 Zincir konumlarını kişi yerine dizgi anahtarıyla gruplama (altı kitap kümesi, yalnız referans etiketleri). İkili precision, aynı anahtara sahip konum çiftleri arasında aynı kişiye ait olanların payı; ikili recall ise aynı kişiye ait çiftler arasında anahtarı ortak olanların payıdır. B-cubed, konumlar üzerinden, aynı anahtarlı konumların aynı kişiye ait olan payını (precision) ve kişinin konumlarının aynı anahtara sahip olan payını (recall) ortalar.

Anahtar	Farklı anahtarlar	Birden fazla kişiyi kapsayan anahtarlar	Bu anahtarların altındaki konumlar	Birden fazla anahtara bölünen kişiler	İkili P / R	B-cubed P / R
Tam olarak yazıldığı biçimiyle ad	964	24 (%2,5)	216 (%10,0)	191 (%25,8)	0,737 / 0,559	0,941 / 0,715
Normalize ad	878	28 (%3,2)	262 (%12,2)	161 (%21,8)	0,759 / 0,703	0,931 / 0,792

Her on zincir konumundan biri, birden fazla kişinin paylaştığı bir adın altında yer alır; kişilerin dörtte biri birden fazla adla görünür. En belirsiz anahtarlar akrabalık ifadeleridir: أَبِيهِ ("babası"), bu örneklemde 34 farklı kişiyi temsil eder. Adın normalizasyonu bölünmeyi azaltır, farklı kişilerin birleştirilmesini artırır. İki anahtar da kimlik değildir. Aynı örüntü AR-Sanad test zincirlerinde de görünür [7]; burada 15.301 yüzey adının 815'i birden fazla kimliğe eşlenir.

Bu, çözümün değil sorunun ölçümüdür. Geçişleri bir otorite listesine bağlamak gelecek çalışmadır.

7.11 Ölçmediklerimiz

- **İnsan altın standardı.** Bu değerlendirmedeki hiçbir hadis iki bağımsız etiketleyici tarafından etiketlenmedi. Etiketleme formu ve kılavuzu vardır; etiketleme yapılmamıştır.
- **İnsan incelemesi.** Hiçbir inceleyici hiçbir aday hakkında karar vermedi. Bu yüzden inceleme süremiz, ret oranımız ve içinde bir kişi bulunan boru hattı ölçümümüz yoktur.
- **İlişkiler.** Ravi geçişlerini ve sıralarını puanladık; `NARRATED_FROM` bağlantılarını üçlüler olarak puanlamadık.
- **Diğer modeller.** Bir model kullanılabilir sonuç üretti. İkinci koşu bir yapılandırma gerekçesiyle başarısız oldu.
- **Şema sapması.** Haiku koşuları 910 çağrıda bir tanımsız özellik ve sıfır tanımsız tip üretti. Bu yüzden sapmaya ilişkin ölçülen kanıt neredeyse bütünüyle başarısız ikinci model koşusundan gelir.
- **Koşular arası değişkenlik.** Her koşul bir kez çalıştırıldı.
- **Ezberleme.** Modelin zincirleri okumak yerine iyi bilinen zincirleri hatırlayıp hatırlamadığını sınamadık.
- **Soru cevaplama.** Graf henüz herhangi bir soru kümesinde metin getirmeyle karşılaştırılmadı.
- **Metin bütünlüğü.** Derlem metnini basılı bir baskıyla karşılaştırmadık.

7.12 Önceki pilot

30 Eylül 2026'daki pilot, aynı 50 hadisi farklı bir modelle (Claude Sonnet 5) motor dışında kullandı. Kural tabanlı ayrıştırıcı için gevşek F1 0,879, yalnız şema içeren istem için 0,865, isnad bilgili istem için 0,935 raporladı; 605 model alıntısının 605'i de kaynakta bulundu; 318 adın 32'si modelin kendi alıntısında

bulunmadı; güven skoru AUC değeri 0,626 oldu. Bu bölümdeki motor tabanlı koşular, farklı bir modelle, altı kat daha fazla görülmemiş veri üzerinde ve farklı bir etiket kaynağına karşı bu örüntülerin her birini tekrar üretir.

8 Destek çalışmaları: neyin saklanması gerekir

Bölüm 7'de motorun kontrollerini ölçtük. Bu bölümde tasarımın diğer yarısı olan karar kaydı üzerine iki küçük çalışmayı raporluyoruz. Bölüm 2.6'da projenin dayandığı iddiayı belirtmiştik. Bir kaynaktan türetilebilen olgu yeniden türetilmelidir; bir kişinin verdiği karar ise türetilemez ve bir kez saklanmalıdır. İlk çalışma, etiketli isnad zincirlerinde kimlik kararını saklamanın ne kazandırdığını sorar. İkinci çalışma aynı soruyu farklı bir alanda, motorun kendi kaynak kodunu okuyan bir kod ajanında sorar.

İki çalışma da keşif amaçlıdır ve 13 Ekim 2026'da gerçekleştirilmiştir. İlki, veri kümesinin etiketlerinin inceleyicinin yerine geçtiği bir benzetimdir. İkincisinde okuyucu olarak bir dil modeli kullanılır. İkisine de hiçbir kişi katılmamıştır; bunun sonuçlarını Bölüm 8.3'te belirtiyoruz.

8.1 Etiketli isnad zincirlerinde kimlik kararının yeniden kullanımı

Bölüm 7.10, yazılı bir adın ravinin kimliği olmadığını gösterdi. Bunu çözen "bu ad, burada, şu kişidir" kararı, motorun kaydettiği türden bir karardır. Böyle bir kararın ne sıklıkta yeniden kullanılabildiğini ve bu kullanımın maliyetini sorduk.

Benzetim. Bir defter, etiketli zincirleri her seferinde bir geçiş okuyarak işler. Daha önce görmediği bir anahtarla karşılaşınca inceleyiciye sorar; yanıt, veri kümesinin etiketidir. Sonrasında aynı anahtar için bu yanıtı yeniden kullanır. İnceleyiciye giden her şey bir *soru* sayılır. Etiketle uyuşmayan ve hiçbir inceleyicinin görmediği, yeniden kullanılmış yanıt ise *sessiz hata* sayılır. İkisini de geçiş başına raporluyoruz. Modül `evaluation/identity_ledger.py`'dır; zincirlerin sırası rastgeledir ve değerler beş sıra üzerinden alınan ortalamalardır.

Veriler. AR-Sanad 280K test zincirleri [7]: 27.056 zincir, 150.904 geçiş, 6.378 kişi ve 15.301 farklı yazılı ad; bu adların 815'i birden fazla kişiyi temsil eder. Tekrar çalışmasında, altı Ifta kitabının ravi kimliği içeren hadislerini kullanıyoruz: 34.681 hadis, 205.514 geçiş ve 6.891 kişi. İki etiket kümesi farklı kaynaklardan gelir [6]. Zincirde bir geçişten önce adı verilen raviyi onun *öğrencisi*, sonra adı verileni ise *hocası* olarak adlandırıyoruz.

Kararın kapsamı. Defter, her yanıtı tam bağlamıyla (öğrenci, ad, hoca) saklar. Yanıt yeniden kullanılmadan önce sonraki bir geçişin bu bağlamın hangi kısmını paylaşması gerektiği, kararın kapsamıdır. Yalnız ad kapsamı, en fazla yeniden kullanımı ve en fazla hatayı verir. Tablo 8.1'de iki sabit kapsamı ve üç daraltma biçimini sunuyoruz.

Daraltma politikalarında kapsam önceden seçilmez. Her ad yalnız ad kapsamıyla başlar; ancak o ad için bir çelişki görüldükten sonra öğrencisine bağlanır. Çelişki algılayıcısı, ravi kataloğu üzerinde çalışan deterministik bir kuraldır: defterin önerdiği kişiyle komşu ravi arasındaki tabaka farkı beşten büyük ya da eksi birden küçükse veya vefat yılları arasındaki fark 85 yıldan büyük ya da eksi 20'den küçükse inceleyiciye sorulur. Sınırlar AR-Sanad zincirlerinde seçilmiştir; burada etiketli komşu çiftlerin %0,8'ini işaretlerler. Ifta verilerine değiştirilmeden uygulanmışlar ve burada %3,4'ünü işaretlemişlerdir.

Tablo 8.1 İki etiket kümesinde beş politika için geçiş başına soru ve sessiz hata (beş rastgele sıra üzerinden ortalamalar). "Sonradan bilme", belirsiz olduğu bilinen adları dar kapsamda başlatır; etiketleri önceden gerektirir ve bir üst sınırdır.

Politika	AR-Sanad: sorular	AR-Sanad: sessiz hatalar	Ifta: sorular	Ifta: sessiz hatalar
Sabit kapsam: ad	0,101	0,100	0,077	0,102

Politika	AR-Sanad: sorular	AR-Sanad: sessiz hatalar	Ifta: sorular	Ifta: sessiz hatalar
Sabit kapsam: ad; kural yalnız işaretlenen geçişi sorar	0,204	0,054	0,229	0,049
Kuralla daralan kapsam	0,178	0,014	0,199	0,010
Sabit kapsam: öğrenci ve ad	0,340	0,006	0,292	0,007
Sonradan bilme	0,187	0,006	0,157	0,007

Yalnız adla anahtarlanan karar, her on geçişten birinde yanlış yeniden kullanılır. Öğrenci ve adla anahtarlamak bu hataların neredeyse tamamını kaldırır, ancak yaklaşık üç katı soru gerektirir. Kapsamı yalnız kuralın çelişki gördüğü yerlerde daraltmak, dar anahtarın hatasına yaklaşır (AR-Sanad'da 0,006'ya karşı 0,014, Ifta'da 0,007'ye karşı 0,010); soru sayısı ise onun yaklaşık yarısı ile üçte ikisi arasındadır.

Aynı kural iki biçimde kullanıldı. Yalnız işaretlenen geçişi inceleyiciye gönderdiğinde hata 0,05 yakınında kalır. İşaretleme aynı zamanda o adın kapsamını daralttığında, bir çelişki adın sonraki bütün geçişlerini korur; hata 0,014 ve 0,010'a düşer.

Defterin görmediği bir kitap. Altı Ifta kitabının beşinden defteri doldurup altıncısında sayım yaptık; bunu her kitap için sırayla tekrarladık. Kuralla daralan kapsamda geçiş başına sessiz hata 0,004 ile 0,016, soru 0,138 ile 0,249 arasındaydı. Öğrenci ve adın sabit kapsamında hata 0,007 ile 0,023, soru 0,151 ile 0,307 arasındaydı; daraltma politikası her kitapta daha az soru sordu. Yalnız adla hata 0,091 ile 0,151 arasındaydı: başka bir kitaptan taşınan yanıt, aynı adı taşıyan başka bir kişiye ait olabilir.

Hata yapan bir inceleyici. Benzetimdeki inceleyici, belirsiz adların %5'i için yanlış yanıt verdiğinde (AR-Sanad), daraltma politikası geçiş başına 0,219 soru ve 0,018 sessiz hata; öğrenci ve adın sabit kapsamı ise 0,340 ve 0,019 verir. Yanlış yanıtlar sahte çelişkiler, dolayısıyla daha fazla soru üretir; politikaların sıralaması değişmez.

Karar, yeniden çıkarımdan sonra da kullanılabilir mi? Saklanan bir karar, ancak metinden ikinci kez çıkarım yapıldığında yeniden bulunabiliyorsa yararlıdır. Motor, kararı bir adaya bağlar. Bölüm 7'deki dört koşunun ravi adaylarını karşılaştırdık; hedef hadis ve normalize ad çifti olduğunda, bir koşunun adaylarının ne kadarının başka bir koşuda karşılığı olduğunu sorduk (`evaluation/decision_reapplication.py`). Aynı modelin iki istemi arasında pay, altı yönde 0,857 ile 0,977 arasındadır. Ayrıştırıcının metin aralıkları uzun kaldığı için kural tabanlı ayrıştırıcı ile bir model koşusu arasında 0,561 ile 0,634 arasındadır. Aday kimliğine bağlandığında pay, yapı gereği sıfırdır. Hiçbir karar verilmediğinden bu, kararların değil hedeflerin örtüşmesini ölçer.

8.2 İkinci bir alan: kod ajanı için neyin saklanması gerekir

İkinci çalışma hadis alanının dışına çıkar. Türetilebilir olgular ile kararlar arasındaki ayrımın, başka bir okuyucu ve başka tür bir kaynak için de geçerli olup olmadığını sorar: bir kod tabanı hakkında soruları yanıtlayan kod ajanı. Kod tabanı, motorun kendi arka ucudur (141 dosya, 16.923 satır); tasarım, sorular ve koşular deneyle birlikte belgelenmiştir [38].

Düzenek. Ajan, düşük çaba ayarında Claude Haiku 5.5'tir ve salt okunur üç aracı vardır: dizin listeleme, dosya okuma ve arama. Üç sınıfın her birinde on iki olmak üzere 36 soruyu yanıtlar. *Değişmeyen* sorular, 5 Nisan 2026 kodunda ve güncel kodda aynı olan yapısal bir olguyu sorar. *Değişen* sorular, Ekim'deki yeniden yazımın değiştirdiği yapısal bir olguyu sorar. *Karar* soruları bir şeyin neden o biçimde olduğunu sorar; yanıtları kodda değil, projenin karar kayıtlarındadır. Sınanan belgeleri görmeyen ayrı bir ajan soruları ve cevap anahtarını yazdı: yapısal sorular için koddan, karar soruları için projenin karar kayıtlarından.

Koşullar yalnız bir `docs/` klasörünün içeriğinde farklılaşır: hiçbir şey; her biri gerekçesini ve reddedilen alternatifi içeren, türetilebilir hiçbir olgu içermeyen 30 kararlık bir defter; defterle birlikte güncel koddan yazılmış, kodu açıklayan bir sayfa (taze sayfa); defterle birlikte aynı talimatlarla 5 Nisan kodundan yazılmış bir sayfa (bayat sayfa). Bir düzende ajan `docs/` klasörünü kendisi açmak zorundadır; diğerinde belgeler isteme de yerleştirilir. Araç çağrısı hakkı 14, 4 ya da 2'dir. Her hücre iki kez çalıştırılmıştır. Yanıtlar, koşulu görmeyen aynı model tarafından yazılı anahtara göre puanlanmıştır. Analiz birimi sorudur; aralıklar sorular üzerinden bootstrap aralıklarıdır. Temel tasarımda 2.880 koşu vardır; aşağıda açıklanan sonraki deney kollarıyla birlikte deney 5.520 koşu içerir.

Tablo 8.2 14 araç çağrısı hakkıyla doğru yanıtların payı (24 yapısal soru ve 12 karar sorusu, her biri için iki koşu).

`docs/` içeriği	Yapısal, ajan `docs/` klasörünü açar	Yapısal, belgeler istemde	Karar, ajan `docs/` klasörünü açar	Karar, belgeler istemde
Hiçbir şey	0,979	0,938	0,042	0,000
Karar defteri	0,938	0,979	1,000	0,958
Defter ve taze sayfa	0,979	0,958	1,000	0,917
Defter ve bayat sayfa	0,896	1,000	0,875	0,958

Yeterli araç çağrısıyla ajan yapıyı koddan okur. Belge olmadan yapısal soruların %94 ile %98'ini yanıtlar ve hiçbir belge türü bunu değiştirmez. `docs/` klasörünü kendisinin açması gereken 432 yapısal koşuda klasörü bir kez bile açmadı. Belgeleri isteme yerleştirmek, yapısal soru başına istem belirteçlerini 21.843'ten 45.343 ile 75.443 arasına çıkardı. Karar sorularında ise durum tersidir. Defter olmadan ajan bunların neredeyse hiçbirini, defterle neredeyse tamamını yanıtlar.

Tablo 8.3 Belgeler istemdeyken ve araç çağrısı hakkı azken yapısal sorular.

`docs/` içeriği	2 çağrı: tamamı	2 çağrı: değişmeyen	2 çağrı: değişen	4 çağrı: tamamı
Hiçbir şey	0,312	0,375	0,250	0,750
Taze sayfa	0,875	0,792	0,958	0,979
Bayat sayfa	0,688	0,875	0,500	0,812

Kodun saklanan açıklaması, yalnız kodu okumak pahalı olduğunda yarar sağlar. Taze sayfa, belgesiz koşula göre doğru yanıt payını iki araç çağrısında 0,56 (aralık 0,38 ile 0,75 arasında), dört çağrıda 0,23 (0,10 ile 0,38 arasında) artırır; on dört çağrıda ise artırmaz. Değişen olgularda bayat sayfa, taze sayfanın yaklaşık yarısı sıklıkta doğrudur.

Saklanan açıklamayı yeniden yazmadan geçerli tutmanın üç yolunu denedik: her bölümün altına, yazıldığı kodun değişip değişmediğini belirten makine üretimi bir not; olguları saklanan seçiciler aracılığıyla güncel koddan doldurulan cümleler; sayfanın yazılmasından sonra eklenen sembollerin listesi. Hiçbiri güncel koddan yeniden yazılmış sayfaya yaklaşmadı. İki çağrıda değişen olgular için doldurulan cümleler 0,625, bayat sayfa 0,500, taze sayfa 0,958 verdi.

Bir ifadenin nereye konduğu, ajana ulaşıp ulaşmadığını belirler. Taze sayfanın tek satırlık notlara bölünüp kaynak ağacına yerleştirilmesi, yalnız arama aracı eşleşen notları koddan önce döndürdüğünde yardımcı olur: iki çağrıda yapısal sorulardaki kazanım, Claude Haiku 5.5 ve Claude Sonnet 5.5 modellerinde 0,15 ile 0,31 arasındadır; dosyalar karıştırılmış sırada arandığında ise 0,04'tür (aralık −0,15 ile 0,23 arasında). Bu kazanımların en küçüğünün aralığı sıfıra ulaşır. Sonnet koşularında notsuz deney kolu, 48 koşunun 5'inde boş yanıt döndürdü. Bu, güvenilir karşılaştırmanın sınırı olarak önceden belirlediğimiz %10'un üzerinde olduğundan bu iki kazanımı bu çekinceyle raporluyoruz.

8.3 İki çalışmanın ekledikleri ve göstermedikleri

İki çalışma, iddianın ilk yarısında Bölüm 2.6 ile uyumludur. Ajan bir kararı koddan elde edemedi; ravinin kimliği de yazılı addan elde edilemedi. Her iki durumda da yanıtın bir kez bir kişiden gelmesi ve saklanması gerekir.

İkinci yarıyı ise sınırlarlar. Türetilebilir bir olgu, kullanım anında türetilmesi ucuzsa saklanmak zorunda değildir; ucuz değilse saklanan kopya yarar sağlar. Ajan için iki araç çağrısında, çıkarımın bir model çağrısı ve inceleme gerektirdiği her yerde durum budur. Yeniden üretilmeyen saklanmış kopya, tam da değişen olgularda yanıltır.

Bir değişken daha eklerler. Saklanan kararın bir kapsamı vardır; kapsamın saklanması gerekmemiştir: algılayıcı olarak deterministik bir alan kuralı kullanılarak çelişkilerden türetilmiştir. Mekanizmanın yeni olduğunu iddia etmiyoruz. Bir düzeltmeyi önceki yanıtın başarısız olduğu bağlama bağlamak, bilgi ediniminde ripple-down rules adıyla bilinir; kaynağın tamamını okumadan bibliyografik kaydından atıf veriyoruz [39].

Sınırlar, benzetimlerin sınırlarıdır. Defter çalışmasında inceleyici veri kümesinin etiketidir; dolayısıyla ilk yanıt ücretsizdir ve %5 varyantı dışında daima doğrudur. Bir soru, kişiden ne kadar zaman isteyeceğine bakılmadan bir birim sayılır. AR-Sanad kişi etiketlerinin nasıl atandığı belgelenmemiştir; altı kitap ravilerinin çoğunu paylaştığı için uzak bir kitap sınanmamıştır. Ajan çalışmasında 17 bin satırlık bir kod tabanı, sınıf başına on iki soru ve hücre başına iki koşu vardır; puanlayıcı ajanla aynı modeldir; sayfalar kişiler değil ajanlar tarafından yazılmıştır; ikinci model ise yalnız iki karşılaştırmada çalışmış ve aynı model ailesinden gelmiştir. İki çalışma da motoru ölçmez ve ikisi de Bölüm 10.1'de planlanan inceleyici deneyinin yerini tutmaz.

9 Tartışma

9.1 Kontrolün yeri

Proje, her olguyu bir kaynak alıntısına bağlamanın kaynağın desteklemediği olguları grafın dışında tuttuğu iddiasıyla başladı. Koşularımızda modelin metinden saptığı yer alıntı değildi. Kontrole ulaşan her alıntı kendi segmentinde bulundu. Farklılaşan, alıntıyla çıkarılan değer arasındaki ilişkiydi: model "babasından" ifadesini alıntılayıp bir ad yazdı ya da Arapça bir adı alıntılayıp Latin harfleriyle yazdı.

Alıntı kontrolü, "model gerçek bir metni mi gösterdi?" sorusunu yanıtlar. Değer-alıntıda kontrolü ise "modelin yazdığı, gösterdiği metinde var mı?" sorusunu yanıtlar. Verilerimizde eşleşen önerileri eşleşmeyenlerden yalnız ikinci soru ayırdı. İki kontrolün de maliyeti bir dizgi karşılaştırmasıdır.

Kontrol, inceleyicinin kabul edeceği değerleri de işaretler. İşaretlenen 155 adın 33'ü referansla eşleşir (%21); bunların 13'ü alıntıdan yalnız ad hâli bakımından farklıdır. Motorun işaretlenen adayı reddetmek yerine incelemeye göndermesinin nedeni budur. Hâl eklerini izin verilen değişiklik olarak tanıyan bir kural, bu 13 adı otomatik kabul eder ve bunu yaptığını kaydeder; diğer 20'yi kaldırmaz.

Kontrol, uydurulmuş raviyle dönüştürülmüş raviyi ayırt etmez. İşaretlenen adların çoğu başka yazı ya da hâlde doğru kişidir; akrabalık örneklerinin çoğu da metnin desteklediği çıkarımlardır. Kontrolün ortaya koyduğu daha dardır ama yine de yararlıdır: model bir şeyi değiştirmiş ve bunu söylememiştir. Olgularının kaynak ifadesine kadar izlenebilmesi gereken bir graf için karar gerektiren olay budur.

9.2 Modele ne için ihtiyaç var?

Gevşek eşleştirmede, model çağrısı yapmayan kural tabanlı ayrıştırıcı F1 0,923'e, yalnız şema verilen model 0,890'a, tip açıklamaları ve alan ipuçları eklenen model 0,956'ya ulaşır.

Ayrıştırıcıyla yalnız şema verilen modelin karşılaştırması, eşleştirme kuralına bağlıdır. Ayrıştırıcı adları bulur ama uzun kalan metin aralıkları döndürür; bu yüzden gevşek eşleştirmede iyi, katı eşleştirmede kötü sonuç verir (0,633). Yalnız şema verilen model temiz adlar döndürdüğü için katı eşleştirmede daha iyidir (0,826) ve daha fazla zinciri bütünüyle doğru çıkarır; ancak ad ekler ve değiştirir. Hangisinin tercih edileceği, sonraki bir adımın metin aralıklarını kırpabilmesine ya da adları düzeltmek zorunda olmasına bağlıdır.

Şema, alan bilgisi değildir. Yalnız şema verilen model tipleri biliyordu ama yine de Peygamber'i ravi saydı ve adları çevriyazıya çevirdi. Şema şablonundaki birkaç açıklama ve talimat cümlesi, eşleşmeyen önerilerin neredeyse tamamını kaldırdı. Bu görevde talimatlar sonucu herhangi bir kontrolden daha fazla değiştirdi. Şemasız talimatları sınamadık.

İpuçları verilen model, çoğunlukla paralel zincirli hadislerdeki recall sayesinde gevşek eşleştirmede ayrıştırıcının 0,034 F1 üzerindedir; katı eşleştirmede fark çok daha fazladır. Bunun hadis başına bir model çağrısına değip değmeyeceği kullanıma bağlıdır. Büyük bir derlemde ilk geçiş için ayrıştırıcının maliyeti yoktur. Motor, aynı boru hattı üzerinden ikisini de destekler.

İstem iyi olduğunda kontroller pek az şey bulur. İstem, model ya da yapılandırma iyi olmadığında grafı koruyan kontrollerdir. İkinci modelle başarısız koşu buna örnektir: çoğu çağrının çıktısı bozuk biçimliydi ve bozuk biçimli hiçbir çıktı commit edilmedi.

9.3 Güven skoru

Bu model ve istemle, modelin güven skoru yönlendirme için zayıf bir temeldi. İncelemeye gönderilen geçişlerin payı karşılaştırılabilir olduğunda, kontrolün bulduğu eşleşmeyen geçişlerin yarısından azını buldu (Tablo 7.8). İlk kod, adayları yalnız güven skoru eşiğine göre incelemeye gönderiyordu. Güncel kod, eşiği korur ve kontrolleri ekler.

9.4 Katı commit kuralının maliyeti

Kontroller değiştirilmiş adları kabul edilen çıktının dışında tutar; commit kuralı ise adaylarından herhangi biri inceleme beklediği sürece hadisin tamamını grafın dışında tutar. Yalnız şema verilen koşulda, bir kişi karar verene kadar 300 hadisin 110'u commit edilmeden kalır ve grafın recall'u 0,582 olur. İyi bir istemle bu maliyet neredeyse kaybolur (300'ün 6'sı). Tasarım işi inceleyiciye taşır; ortadan kaldırmaz. Bu işin ne kadar sürdüğünü ölçmedik.

9.5 Saklanan kararlar ve türetilen olgular

Motor iki tür şeyi ayrı tutar. Adaylar türetilir: aynı metinden yeniden çıkarım yapılabilir ve Bölüm 7 bu türetimin kontrolü hakkındadır. Kararlar türetilmez: kabul, düzeltme ya da kimlik metinden elde edilemez; bu yüzden motor bunları bir kez kaydeder ve hiçbir zaman üzerine yazmaz. Bölüm 8'de ikinci türü iki yerde sınadık; ikisinde de sonuç aynıydı. Kod ajanı, kodun neden o biçimde olduğunu koddan yanıtlayamadı; karar defteriyle yanıtlayabildi. Ravinin kimliği yazılı addan okunamadı; saklanan kimlik kararı yeniden kullanılabildi.

Bundan motor için iki sonuç çıkar. Birincisi, saklanan kararın bir kapsama ihtiyacı vardır; kapsam geniş başlayıp çelişki ortaya çıktığında daralabilir. Benzetimde, tabaka ve vefat yılları üzerine tek bir deterministik kural, yaklaşık yarısı kadar soruyla sessiz hatayı dar anahtarın hatasına yaklaştırmaya yetti. Motorda böyle bir kural yoktur (biyografik tarihler gerektiren kurallar kapsam dışıdır, Bölüm 4); kararlar da ikinci çıkarımın değiştirdiği adaylara bağlıdır. Bölüm 8.1'deki ölçüm, bunun yerine içeriğe bağlansalar ne kadarının taşınacağını söyler: bir modelin iki istemi arasında çoğu, ayrıştırıcı ile model arasında üçte ikisinden azı.

İkincisi, iddianın diğer yarısını sınırlar. "Yeniden türet", türetmenin maliyetini karşılayabilen okuyucuya verilen bir öneridir. On dört araç çağrısı olan ajan, kodun açıklamasına ihtiyaç duymadı; iki çağrısı olduğunda duydu. Yeniden üretilmemiş açıklama, değişen olgularda onu yanılttı. Motordaki çıkarım pahalı durumdur; çünkü her türetim bir model çağrısı ve olası bir inceleme gerektirir. Commit edilen grafın bir depo olarak tutulmasının nedeni budur. İçindeki her olgunun nasıl kabul edildiğine ilişkin kaydı taşımasının nedeni de, ona güvenmek yerine yeni bir çıkarıma karşı kontrol edilebilmesidir.

9.6 Geçerlilik tehditleri

Referans etiketleri. Ifta etiketleri, üçüncü bir tarafın yeniden yapılandırdığı platform işaretlemeleridir; gözlemlediğimiz ve düzeltmediğimiz tutarsızlıklar içerir. Sistem, platformun işaretlemediği bir raviyi çıkardığında bazen cezalandırılır. Farklı biçimlerde üretilmiş iki etiket kümesi, ortak hadislerinde birbirine yakındır (F1 0,959); bu, etiketlerin koşulları karşılaştırmak için yeterli olduğunu düşündürür. İki kümenin ne kadar bağımsız olduğunu bilmiyoruz; mutlak değerler doğruluk olarak okunmamalıdır.

Eşleştirme kuralı. Gevşek eşleştirme fazladan sözcük içeren geçişi kabul eder ve aynı hâl değişikliğini adın uzunluğuna göre eşleşme ya da eşleşmeme sayar. Bir kuralda geçerli olup diğerinde olmayan sonuçları, kurallarıyla birlikte belirtiyoruz.

"Yanlış" ne demektir? Eşleşmeyen öneri, mutlaka hadis hakkında yanlış bir ifade değildir. Yalnız şema verilen modelin eşleşmeyen önerilerinin yarısından fazlası, doğru raviyi başka bir yazıyla adlandırır.

Tek alan. Değer-alıntıda kontrolünün eşleşmeyen önerilerin çoğunu (145'in 122'sini) işaretlemesi, modelin metinden başlıca sapmasının adı yeniden yazmak olduğu bu görevin ve bu istemin özelliğidir. Kontrolü hadis dışında, kimyasal madde ve hastalık geçişleri için insan etiketleri içeren BC5CDR derleminin 300 özetinde [40], aynı model ve yalnız şema içeren bir istemle tekrarladık. Burada kontrol 2.269 önerinin %3,9'unu işaretledi; bunların içinde etiketlerle eşleşmeyen 303 önerinin 53'ü vardı. Diğer 250'si, etiketleyicilerin etiketlemediği metin aralıklarının aynen kopyalarıydı. Aynı sayıda öğe incelemeye gönderildiğinde modelin güven skoru eşleşmeyen 58 öneri, kontrol ise 53 öneri buldu. Kontrol, değerin alıntılanan metinden geldiğini ortaya koyar. Doğru metnin seçildiğini ortaya koymaz.

Tek model, tek çaba ayarı, tek koşu. Kullanılabilir bütün sonuçlar düşük çaba ayarındaki Claude Haiku 5.5'ten gelir; her koşul bir kez çalıştırılmıştır. İkinci model, kendi aktarım biçimimize ve çaba ayarımıza bağladığımız bir nedenle başarısız oldu; bu açıklamayı sınamadık.

Bellek içi arka uç. Deneyler bellek içi arka uçta çalıştı. Aynı boru hattı PostgreSQL ve Neo4j üzerinde deneylerle değil, on iki testle kapsanır.

Ayarlama. Kural tabanlı bölütleyici 50 pilot hadis üzerinde ayarlandı. Altı kitap kümesi bunları dışarıda bıraktığı için ana sonuçlar etkilenmez; ancak ayrıştırıcının kuralları, Bukhari'yi içermeyen pilotta görülen kitapları yansıtır.

Ezberleme. Hadisler iyi bilinir ve modelin eğitim verilerinde bulunmaları muhtemeldir. Metnin hiçbir biçimde içermediği işaretlenmiş bir ad bulduk; bu, modelin zincirleri hatırlayıp hatırlamadığını söylemek için çok azdır. Bunu değiştirilmiş adlarla sınamadık.

Ölçülen döngüde insan yok. İnceleme kuyruğu, karar kaydı ve commit korumaları testlerle kapsanır; ancak değerlendirme inceleyici olmadan çalıştı. Motorun döngüde bir kişiyle hataları azalttığı iddiasını burada ölçmedik. Aynısı Bölüm 8 için de geçerlidir: defter çalışmasının inceleyicisi veri kümesinin etiketi, ikinci çalışmanın okuyucusu ise bir dil modelidir.

Şema sapması. Başlık şema sapmasını içerir. Haiku 5.5'te neredeyse hiç sapma gözlemlemedik; ikinci modelde gözlemlediğimiz ise başarısız bir yapılandırmadan geldi. Motorun sapmayı ele alışını test ettik; normal kullanımdaki sıklığı belirlenmiş değildir.

Kodun ve analizin yazarlığı. Değerlendirilen daldaki kodun çoğu, analiz betikleri ve bu metnin ilk taslağı, yazarların yönlendirmesiyle yapay zekâ kodlama asistanları kullanılarak üretildi. Ölçümler, önbelleğe alınmış model yanıtlarından ve betiklerden yeniden üretilebilir. Yazarlar her ifadenin sorumluluğunu taşır.

9.7 Sistemin sınırları

Ravi düğümü, yazılı adıyla tanımlanır. Bölüm 7.10, bunun yol açtığı hatanın referans etiketlerindeki büyüklüğünü gösterir: böyle anahtarlanan graf, kısa bir adı paylaşan farklı kişileri birleştirir ve bir kişiyi farklı yazımlara böler. Güncel graf üzerine kurulan her ravi ağı analizi bunu devralır. Bölüm 8.1'deki defter, etiketleri yeniden oynatan bir değerlendirme modülüdür; motorun parçası değildir.

Rivayet metni başladıktan sonra başlayan zincirler bölütlenmez; birlikte anılan ortak raviler tek geçişte kalır. İkisi de bölütleyiciyi kullanan koşulların recall'unu düşürür.

Hiçbir kontrol, Peygamber'in ravi olarak adlandırılması gibi kategori hatalarını kapsamaz. Kontroller, elle girilen adaylara değil, çıkarım çağrısından gelen adaylara uygulanır. Kabul kayıtları, istendiğinde saklanan kararlardan ve doğrulama sonuçlarından hesaplanır. Commit edilen koşuyu geri alma gerçekleştirilmemiştir.

10 Sonuç ve gelecek çalışma

Bir dil modeliyle bilgi grafı arasına deterministik kontroller ve kayıtlı bir kabul yerleştiren bir motor kurduk; bu yerleşimin hadis ravi zincirlerinde ne sağladığını, tek model ve platform kaynaklı etiketlerle ölçtük. Motor, böyle bir sistemin saklaması gereken şeyin bir kişinin kararı olduğu ve kaynaktan türetilebilenin yeniden türetildiği iddiasına dayanır. İki destek çalışmasıyla bu iddiayı sınadık.

Ölçümler, bu sınırlar içinde dört ifadeyi destekler. Modelin alıntısıyla çıkardığı değer arasındaki ilişkiyi kontrol etmek, talimatları zayıf verilmiş bir modelin referansla eşleşmeyen önerilerinin çoğunu ek model çağrısı olmadan işaretler; yalnız alıntıyı kontrol etmek ise hiçbirini işaretlemez. Yalnız şema içeren isteme açıklamalar ve alan talimatları eklemek, çıkarımı herhangi bir kontrolden daha fazla iyileştirir. Karşılaştırılabilir bir inceleme hakkıyla modelin güven skoru, bu önerilerin kontrolden çok daha azını bulur. Yazılı ad da ravinin kimliği değildir: her on zincir konumundan biri dizgi düzeyinde belirsizdir.

Destek çalışmaları, ikisi de benzetimlerden gelen iki ifade ekler. Kimlik kararı saklamaya değerdir ve kapsamı sonucu etkiler: yalnız adla anahtarlandığında her on geçişten birinde yanlış yeniden kullanılır. Deterministik kuralın çelişki gördüğünde daralttığı kapsam ise iki etiket kümesinde ve dışarıda bırakılan kitaplarda, yaklaşık yarısı ile üçte ikisi kadar soruyla dar anahtarın hatasına yaklaşır. İkinci bir alanda da okuyucu, kaynaktan kararları elde edememiş ama yapıyı elde edebilmiştir; bu yüzden kaynağın saklanan açıklaması, yalnız kaynağı okumak pahalı olduğunda yarar sağlamıştır.

Ölçümler, motorun ya da defterin bir inceleyiciyle nasıl sonuç verdiğini, diğer modellerle sonuçların nasıl değiştiğini, şema sapmasının ne sıklıkta oluştuğunu veya bağımsız insan etiketleyicilerin ürettiği etiketlere karşı çıktının ne kadar doğru olduğunu göstermez.

10.1 Bitirme Projesi II planı

1. **İnsan etiketli referans.** İki etiketleyici, mevcut form ve kılavuzla 50 pilot hadisi etiketler. Uyumu ve hadis başına süreyi ölçer, kılavuzu düzeltir, sonra altı kitap kümesinden bir örneklemi etiketleriz. Hadis sayısını ölçülen süre belirler.
2. **Döngüde inceleyici.** Hadisi etiketlememiş bir inceleyici, panelde o hadisin inceleme öğeleri hakkında karar verir. İnceleme sonrası precision ve recall'u, gerekçe kodu başına ret oranını ve karar kaydından karar başına saniyeyi raporlarız.
3. **Ravi kimliği.** Geçişleri Ifta verilerinin ravi kataloğuna bağlarız; belirsiz adlarda sıralanmış adayları inceleyiciye sunar, yanlış birleştirmeleri kaçırılan birleştirmelerden ayrı raporlarız. Kararları adaylar yerine içeriğe bağlarız; kimlik kararlarını Bölüm 8.1'deki gibi kapsamı çelişkide daralan bir defterde saklarız. Bu ölçümü, etiketler yerine gerçek bir inceleyicinin kararlarıyla tekrarlarız.
4. **Dönüşüm kuralları ve kategori kuralı.** Ad hâli değişikliklerini otomatik kabul eder ve kabul eden kuralı kaydederiz; böylece değer kontrolünün işaretlediği eşleşen 33 adın 13'ü incelemeden çıkar. Peygamber gibi ravi olmayan adlar için bir kural ekleriz.
5. **Modeller ve aktarım biçimi.** Özellik listesinin alanlarını yeniden adlandırırız; sonra en az biri açık model olmak üzere en az iki modelle daha, tekrarlı koşular yaparak değerlendirmeyi yineleriz. Ezberlemeyi sınamak için değiştirilmiş adlar da kullanırız.
6. **İlişkiler ve zincirler.** `NARRATED_FROM` bağlantılarını ve zincirlerin bütününü puanlarız; rivayet metninden sonra başlayan zincirleri bölütler, birden fazla zincirli hadislerde zincir başına çağrıları sınarız.
7. **Graf üzerinde sorular.** Küçük bir yapısal soru, yol sorusu ve karar sorusu kümesi oluşturur; graftan gelen yanıtları metin getirmeden gelen yanıtlarla karşılaştırırız.

10.2 Hadis dışındaki alanlar

Kontroller, inceleme kuyruğu, karar kaydı ve ilişki kimliği her şemada çalışır; hadise özgü davranışların çoğu bir bölütleme stratejisinde, bir şablonda ve bir sağlayıcıda yer alır. Motorun üç yerinde hâlâ vaka çalışmasının adı geçer (Bölüm 5.1). Aynı yapının başka bir alana hizmet ettiği iddiası motor için sınanmamıştır; bunu sınamanın yolu ikinci bir vaka çalışmasıdır. Hadis dışında elde ettiklerimiz kısmidir. Kararlar ile türetilebilir olgular arasındaki ayrım, tek kod tabanında bir kod ajanı için geçerli kaldı (Bölüm 8.2). Değer-alıntıda kontrolü, biyomedikal özetlere aynı biçimde taşınmadı (Bölüm 9.6). Komşularıyla anahtarlanan bir karar da ancak ravi zincirlerinde olduğu gibi bağlamın kendisi tekrarlandığında yeniden kullanılabilir.

Proje geliştirilirken kaynak kod özel bir depoda tutulmaktadır; çünkü aynı depo üçüncü taraflara ait hadis metinleri ve yayımlanmamış taslaklar içerir. Motoru proje sonunda açık kaynak lisansıyla yayımlamayı planlıyoruz.

Kaynaklar

[1] J. M. Oumer, "Relation extraction for graph-based representation of Hadith Isnad," M.S. thesis, University of Malta, Faculty of Information and Communication Technology, Department of Artificial Intelligence, 2025. https://www.um.edu.mt/library/oar/handle/123456789/137629

[2] A. B. Kamran, B. Abro, and A. Basharat, "SemanticHadith: An ontology-driven knowledge graph for the hadith corpus," *Journal of Web Semantics*, vol. 78, pp. 100797, 2023. doi:10.1016/j.websem.2023.100797

[3] A. B. Kamran, N. A. Butt, and A. Basharat, "Semantic Enrichment of Hadith Corpus—Knowledge Graph Generation From Islamic Text," *Semantic Web*, vol. 17, no. 2, pp. 1–27, 2026. doi:10.1177/22104968261431425

[4] T. Wirahman, "Linking Hadith Narrator Identities Across Heterogeneous Arabic Biographical Databases: A Multi-Signal Entity Resolution Pipeline," arXiv:2607.05424, 2026. https://arxiv.org/abs/2607.05424

[5] M. Mghari, O. Bouras, and A. El Hibaoui, "Sanadset 650K: Data on Hadith narrators," *Data in Brief*, vol. 44, pp. 108540, 2022. doi:10.1016/j.dib.2022.108540

[6] GiraffeGraph project, data manifest: sources, licence statements and construction of the evaluation sets, `data/README.md`, 12 October 2026 (unpublished).

[7] S. Mahmoud, O. Saif, E. Nabil, M. Abdeen, M. ElNainay, and M. Torki, "AR-Sanad 280K: A Novel 280K Artificial Sanads Dataset for Hadith Narrator Disambiguation," *Information*, vol. 13, no. 2, pp. 55, 2022. doi:10.3390/info13020055

[8] M. Aslan, "Tabakat Kitapları ve Hadis İlmine Sağladıkları Faydalar," *Bingöl Üniversitesi İlahiyat Fakültesi Dergisi*, vol. 5, no. 10, pp. 269–290, 2017.

[9] M. A. Haque and R. Islam, "Hadith computational science in the age of large language models: a critical narrative review," arXiv:2608.20364, 2026. https://arxiv.org/abs/2608.20364

[10] A. Hogan et al., "Knowledge Graphs," *ACM Computing Surveys*, vol. 54, no. 4, pp. 1–37, 2021. doi:10.1145/3447772

[11] M. Kejriwal, *Domain-Specific Knowledge Graph Construction*. Springer International Publishing, 2019. doi:10.1007/978-3-030-12375-8

[12] H. Ye, N. Zhang, H. Chen, and H. Chen, "Generative Knowledge Graph Construction: A Review," in *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, pp. 1–17, 2022. doi:10.18653/v1/2022.emnlp-main.1

[13] S. Pan, L. Luo, Y. Wang, C. Chen, J. Wang, and X. Wu, "Unifying Large Language Models and Knowledge Graphs: A Roadmap," *IEEE Transactions on Knowledge and Data Engineering*, vol. 36, no. 7, pp. 3580–3599, 2024. doi:10.1109/tkde.2024.3352100

[14] B. Zhang and H. Soh, "Extract, Define, Canonicalize: An LLM-based Framework for Knowledge Graph Construction," in *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pp. 9820–9836, 2024. doi:10.18653/v1/2024.emnlp-main.548

[15] Z. Ji et al., "Survey of Hallucination in Natural Language Generation," *ACM Computing Surveys*, vol. 55, no. 12, pp. 1–38, 2023. doi:10.1145/3571730

[16] N. Mihindukulasooriya, S. Tiwari, C. F. Enguix, and K. Lata, "Text2KGBench: A Benchmark for Ontology-Driven Knowledge Graph Generation from Text," arXiv:2308.02357, 2023. doi:10.48550/arXiv.2308.02357

[17] S. Min et al., "FActScore: Fine-grained Atomic Evaluation of Factual Precision in Long Form Text Generation," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pp. 12076–12100, 2023. doi:10.18653/v1/2023.emnlp-main.741

[18] J. Wei et al., "Long-form factuality in large language models," in *Advances in Neural Information Processing Systems 37*, pp. 80756–80827, 2024. doi:10.52202/079017-2567

[19] H. Rashkin et al., "Measuring Attribution in Natural Language Generation Models," *Computational Linguistics*, pp. 1–66, 2023. doi:10.1162/coli_a_00490

[20] P. Laban, T. Schnabel, P. N. Bennett, and M. A. Hearst, "SummaC: Re-Visiting NLI-based Models for Inconsistency Detection in Summarization," *Transactions of the Association for Computational Linguistics*, vol. 10, pp. 163–177, 2022. doi:10.1162/tacl_a_00453

[21] Y. Zha, Y. Yang, R. Li, and Z. Hu, "AlignScore: Evaluating Factual Consistency with A Unified Alignment Function," in *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 11328–11348, 2023. doi:10.18653/v1/2023.acl-long.634

[22] L. Gao et al., "RARR: Researching and Revising What Language Models Say, Using Language Models," in *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 16477–16508, 2023. doi:10.18653/v1/2023.acl-long.910

[23] S. Dhuliawala et al., "Chain-of-Verification Reduces Hallucination in Large Language Models," in *Findings of the Association for Computational Linguistics ACL 2024*, pp. 3563–3578, 2024. doi:10.18653/v1/2024.findings-acl.212

[24] S. Kadavath et al., "Language Models (Mostly) Know What They Know," arXiv:2207.05221, 2022. https://arxiv.org/abs/2207.05221

[25] M. Xiong et al., "Can LLMs Express Their Uncertainty? An Empirical Evaluation of Confidence Elicitation in LLMs," arXiv:2306.13063, 2023. https://arxiv.org/abs/2306.13063

[26] Y. E. Kuruçay, "The Cache and the Ledger," project essay, GiraffeGraph workspace, `data/cache-and-ledger/README.md`, 2026 (unpublished).

[27] A. N. Angelopoulos and S. Bates, "A Gentle Introduction to Conformal Prediction and Distribution-Free Uncertainty Quantification," arXiv:2107.07511, 2021. https://arxiv.org/abs/2107.07511

[28] A. Z. Raja, "Grading the Narrators: An Isnad-Rijal Framework for Claim-Level Provenance in Multi-Agent Knowledge Systems," arXiv:2607.24117, 2026. https://arxiv.org/abs/2607.24117

[29] "Sanadset 650K (Hugging Face mirror, CC0-1.0)," 2022.

[30] S. Altammami, E. Atwell, and A. Alsalka, "Text Segmentation Using N-grams to Annotate Hadith Corpus," in *Proceedings of the 3rd Workshop on Arabic Corpus Linguistics*, pp. 31–39, 2019. https://aclanthology.org/W19-5605/

[31] R. Muther and D. Smith, "Tracing Traditions: Automatic Extraction of Isnads from Classical Arabic Texts," in *Proceedings of the Fifth Arabic Natural Language Processing Workshop*, pp. 130–138, 2020. https://aclanthology.org/2020.wanlp-1.12/

[32] M. Asgari-Bidhendi, M. A. Ghaseminia, A. Shahbazi, S. A. Hossayni, N. Torabian, and B. Minaei-Bidgoli, "Rezwan: Leveraging Large Language Models for Comprehensive Hadith Text Processing: A 1.2M Corpus Development," arXiv:2510.03781, 2025. https://arxiv.org/abs/2510.03781

[33] O. Çinici, "Hadis Araştırmalarında Büyük Dil Modelleri (LLM): Teorik Çerçeve, Sınırlar ve Yapay Zekâ Destekli Yapılandırılmış Veri İnşası," *Mevzu: Sosyal Bilimler Dergisi*, no. 16, pp. 299–326, 2026.

doi:10.56720/msbd.2026.s16.326

[34] B. Mo et al., "KGGen: Extracting Knowledge Graphs from Plain Text with Language Models," in *Advances in Neural Information Processing Systems 38*, pp. 33962–33985, 2025. doi:10.52202/085713-1010

[35] Y. Lairgi, L. Moncla, R. Cazabet, K. Benabdeslem, and P. Cléau, "iText2KG: Incremental Knowledge Graphs Construction Using Large Language Models," in *Web Information Systems Engineering – WISE 2024*, pp. 214–229, 2024. doi:10.1007/978-981-96-0573-6_16

[36] J. Gao, X. Li, Y. E. Xu, B. Sisman, X. L. Dong, and J. Yang, "Efficient knowledge graph accuracy evaluation," *Proceedings of the VLDB Endowment*, vol. 12, no. 11, pp. 1679–1691, 2019. doi:10.14778/3342263.3342642

[37] A. Mhamedi, M. Mghari, and A. El Hibaoui, "Computational methodologies for sanad-based hadith analysis: a review," *TELKOMNIKA (Telecommunication Computing Electronics and Control)*, vol. 24, no. 3, pp. 840–851, 2026. doi:10.12928/telkomnika.v24i3.27447

[38] GiraffeGraph project, agent question-answering experiment: design, questions, runs and analysis, `data/cache-and-ledger/data/agent-qa/README.md`, 13 October 2026 (unpublished).

[39] P. Compton and R. Jansen, "A philosophical basis for knowledge acquisition," *Knowledge Acquisition*, vol. 2, no. 3, pp. 241–258, 1990. doi:10.1016/s1042-8143(05)80017-2

[40] J. Li et al., "BioCreative V CDR task corpus: a resource for chemical disease relation extraction," *Database*, vol. 2016, pp. baw068, 2016. doi:10.1093/database/baw068

Ek A Sonuçları yeniden üretme

Bir yol aksi yönde belirtmedikçe bütün komutlar proje deposundaki `giraffegraph-core/` dizininden çalıştırılır. Depo geliştirme sırasında özeldir; motor proje sonunda yayımlanacaktır. Değerlendirme verileri `data/README.md` içinde açıklanmıştır ve yeniden dağıtılmamaktadır.

Veritabanı ya da ağ gerektirmeyen arka uç testleri:

```
cd backend
uv sync --extra dev
uv run pytest -q
```

Gerçek veritabanları üzerindeki testler, bir PostgreSQL 16 ve bir Neo4j 5 örneği ile üç ortam değişkeni gerektirir:

```
GG_TEST_POSTGRES_DSN=... GG_TEST_NEO4J_URI=... GG_TEST_NEO4J_PASSWORD=... uv run pytest -q -m real_db
```

Bir değerlendirme koşusu ve analizi:

```
cd backend
uv run python scripts/run_isnad_experiment.py --input <hadiths.json> --out-dir <run-dir> \
    --condition llm_isnad_whole --model claude-haiku-5-5 --effort low --max-cost-usd 0.85
cd ..
python evaluation/analyze_experiment.py --run <run-dir> --reference <reference.jsonl> --out <run-
dir>/analysis
python evaluation/compare_runs.py --runs <run-dir> <run-dir> ... --reference <reference.jsonl> --out
compare.md
python evaluation/extra_tables.py --runs <run-dir> ... --reference <reference.jsonl> \
    --reference-chains <reference_chains.jsonl> --out extra-tables.json
python evaluation/classify_value_flags.py --run <run-dir> --hadiths <hadiths.json> \
    --reference <reference.jsonl> --out value-flag-classes.json
python evaluation/identity_baseline.py --reference-chains <reference_chains.jsonl> --out identity.json
```

Model yanıtları `backend/var/llm-cache/` altında önbelleğe alınır. Önbellek bulunduğunda koşu hiçbir API çağrısı yapmaz ve aynı çıktıyı yeniden üretir.

Bölüm 8'in destek çalışmaları. Defter benzetimi ve yeniden çıkarımdaki örtüşme, `evaluation/README.md` içinde açıklandığı biçimde çalıştırılır (`identity_ledger.py`, `decision_reapplication.py`). Ajan deneyi, proje deposundaki `data/cache-and-ledger/data/agent-qa/` dizininden çalıştırılır; README'si komutları, soruları, cevap anahtarını ve her deney kolunun maliyetini verir.

Ek B Sonuç dosyaları

İçerik	`giraffegraph-core/evaluation/results/` altındaki yol
Altı kitap test kümesi, dört koşul	`2026-10-12-sixbooks300/` (`compare-ifta.md`, `extra-tables-ifta.json`, koşu başına bir klasör; `haiku-llm_schema_whole/value-flag-classes.json`)
Pilot küme, dört koşul	`2026-10-12-pilot50/` (`compare-sanadset.md`)
İkinci model alt kümesi	`2026-10-12-subset36/` (`compare-ifta.md`)
Ravi kimliği taban çizgisi	`2026-10-12-identity-baseline/sixbooks300.json`
İlk uçtan uca koşunun kural tabanlı taban çizgisi	`2026-10-12-rule-based-baseline/`
Kimlik defteri benzetimi ve yeniden çıkarım örtüşmesi (Bölüm 8.1)	`2026-10-13-identity-ledger/` (`identity-ledger.json`, `decision-reapplication.json`)

Ajan deneyinin koşuları ve analizi (Bölüm 8.2), bu klasörün dışında, `data/cache-and-ledger/data/agent-qa/` içindedir (`analysis.json`, `runs.tar.gz`).

Her koşu klasörü, puanlayıcının biçimindeki sistem çıktısını, durum ve gerekçe kodlarıyla aday başına bir satırı, belirteç ve gecikmeyle sağlayıcı çağrısı başına bir satırı, koşu raporunu ve analizi tutar.

Ek C Yapay zekâ araçlarının kullanımı

Bu proje boyunca yapay zekâ asistanları kullanıldı: literatür taramasında, 30 Eylül 2026'daki tasarım analizinde ve pilotta, değerlendirilen dalda yazılan kodun çoğunda, analiz betiklerinde ve bu raporun taslağını hazırlarken. Çıkarım deneylerinin kendisi de sınanan sistem olarak bir dil modeli kullanır.

Bu çalışma biçiminin risklerini sınırlamak için aşağıdaki kuralları uyguladık. Bibliyografik kayıtlar Crossref, arXiv, ACL Anthology ya da yayıncının ilk sayfasından alındı; hiçbir zaman bellekten yazılmadı. Önceki bir proje belgesindeki üç kaynak hiçbir dizinde bulunamadı ve burada kullanılmadı. Bu rapordaki her ölçülmüş sayı bir betik tarafından üretildi ve bir sonuç dosyasında saklanıyor; veri kümelerinin istatistikleri veri manifestosundan alındı. Kod yalnız testlerle kabul edildi; gerçek veritabanları için işaretlenen testler PostgreSQL ve Neo4j üzerinde çalıştırıldı. Bu raporun bir taslağı, bağımsız bir yapay zekâ inceleyicisi tarafından sonuç dosyalarına karşı kontrol edildi; rapor, dosyaların gösterdiğinden fazlasını iddia ettiği yerlerde düzeltildi. Diğer çalışmalar hakkındaki ifadeler, atıf verilen kaynakta okunanlarla sınırlıdır.

Yazarlar bu raporun içeriğinden sorumludur.